Schedule

- Mon 14 Sep, FTC06 Self Checking✓
- Mon 21 Sep, No Class
- Mon 28 Sep, FTC07 Fault Masking
- Mon 05 Oct, Review
- Mon 12 Oct, Midterm (Fault Tolerant Computing)
  - 13.30-16.30 -TBD-
  - Open any Books including notebooks!!
- Mon 19 Oct, TDC01 Overview of Testing Digital Circuits
- Mon 26 Oct, TDC02 Delay Fault Testing
- Mon 02 Nov, TDC03 Digital Circuit Testing Issues
- Mon 09 Nov, TDC04 Asynchronous Circuit Testing
- Mon 16 Nov, Review

Fault Tolerant System Implementation (Self Checking Circuits)

2110473 Fault Tolerant Computing

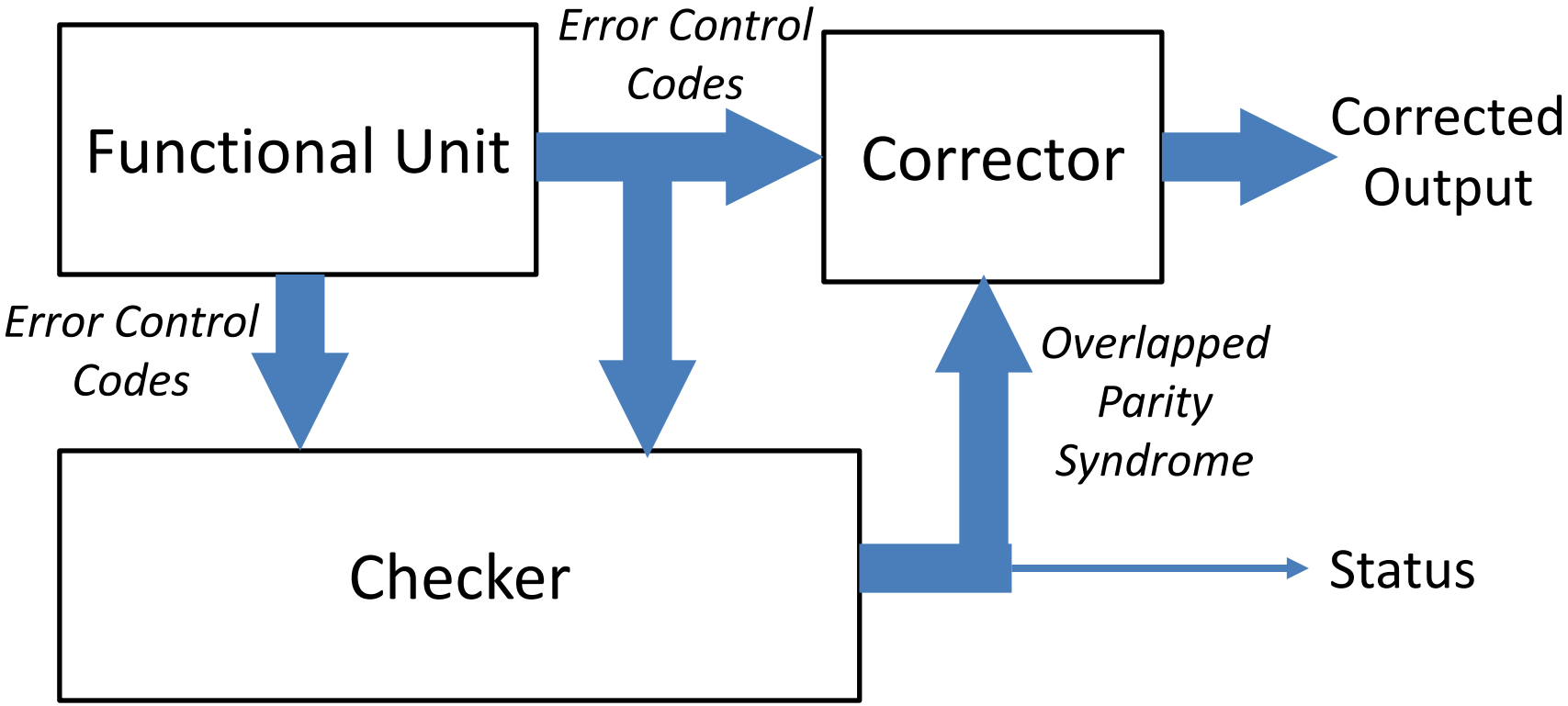
Eiji Fujiwara, Code Design for Dependable Systems, JOHN WILEY & SONS, 2006

Contents

- Self Checking Concept
- Fault Secure and Self Testing
- Totally Self Checking and Strongly Fault Secure
- Error Secure and Error Preserving
- Strongly Code Disjoint
- Self Checking Logic Network
- Types of Checker
- Self Checking Checker
- Implementations of Checker
- Implementations of Functional Units

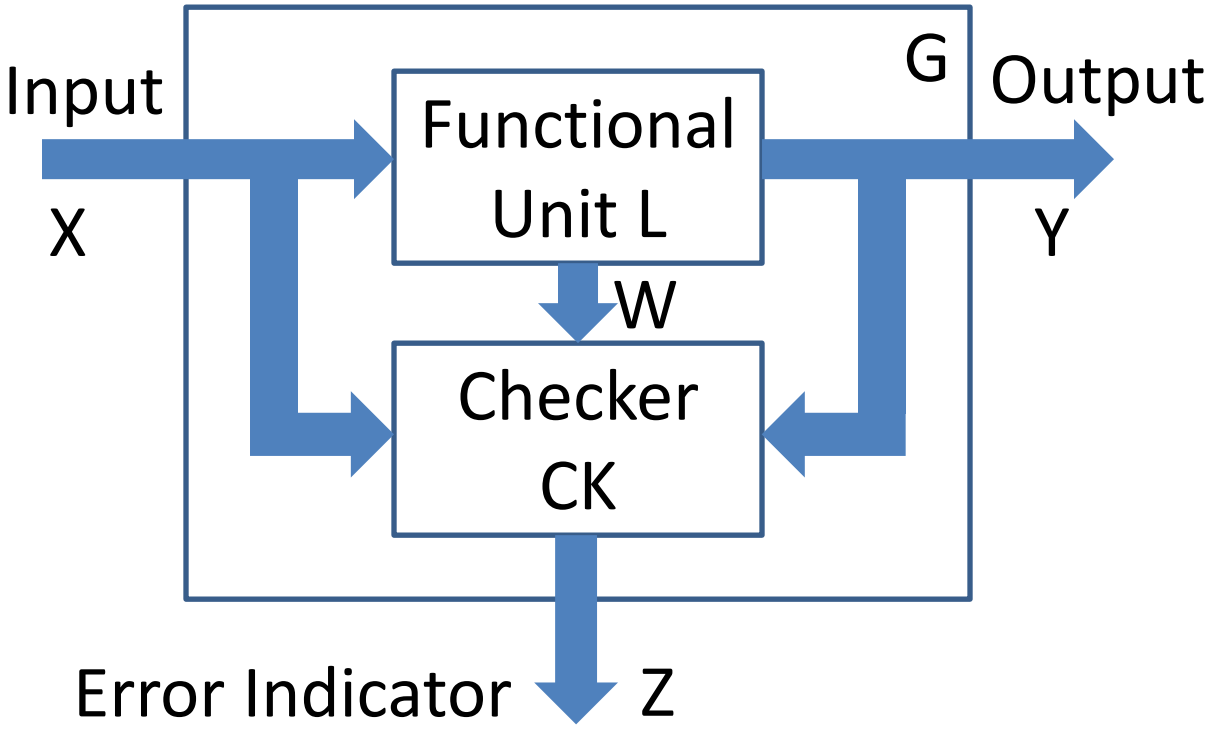
Error Control

- Fault Masking Scheme



Self Checking Model

- Concurrent Fault/Error Detection Scheme



Self Checking Concept

- Self Checking is an approach for Concurrent error detection (CED) that allows:
  - detection of both temporary and permanent faults
  - early detection of errors so that corrective action can be initiated before data corruption
  - easy identification of the field replaceable unit (eg. a chip or a board)
- The use of a CED scheme in order to achieve the high reliability requirement of modern systems is becoming an important design technique

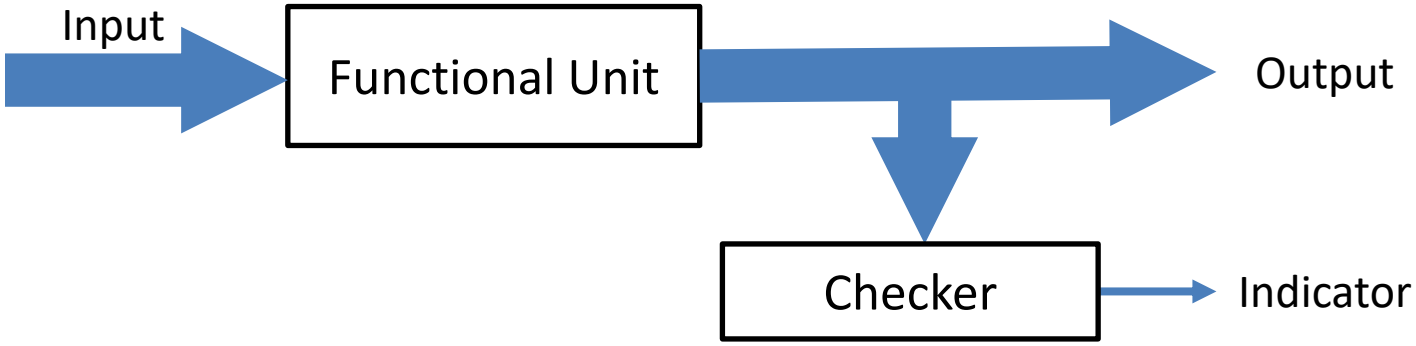
Error Detecting Codes

- Design of Self Checking logic circuits is based on error detecting codes
- Error detecting codes - specific presentation of symbols which enables error detection inside codeword
- Most frequently used error detecting codes:
  - **Separable:**– *Parity code, Berger Code*
  - **Non-Separable:**–m-of-n *Code, Dual Rail Code (Two Rail Code)*

Basic Idea of Self Checking

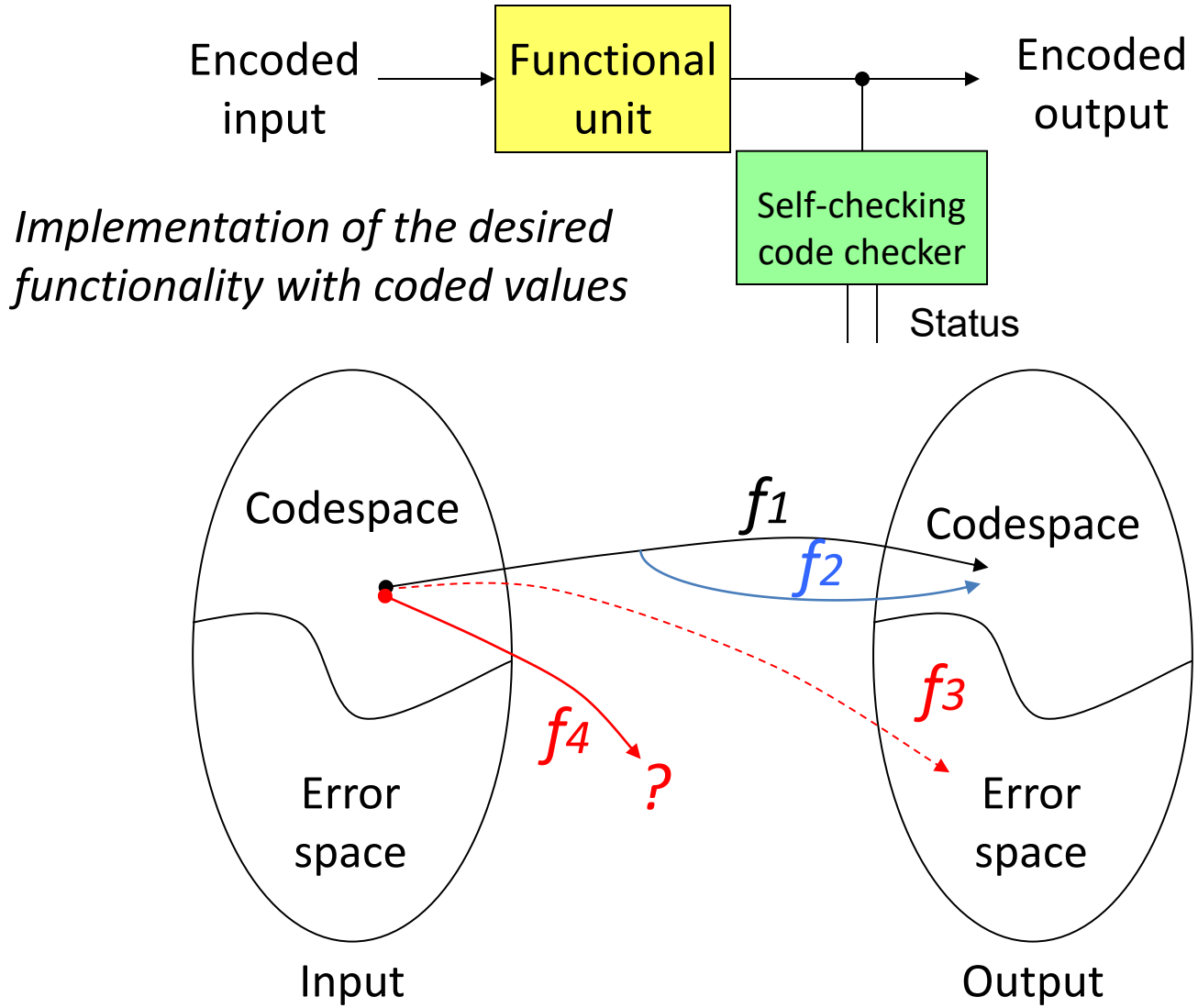
- Digital circuit has a self checking property if it is able to detect internal faults immediately after its occurrence, without using external test signals
- Information redundancy (Error Control Coding), which require an additional hardware, is introduced
  - The outputs of function units must satisfy specific features
- Function unit designed in a way that faults/errors/malfunctions manifest themselves as invalid (error-space) outputs, which are detectable by an external code checker

Four possibilities of Functional Unit & Checker

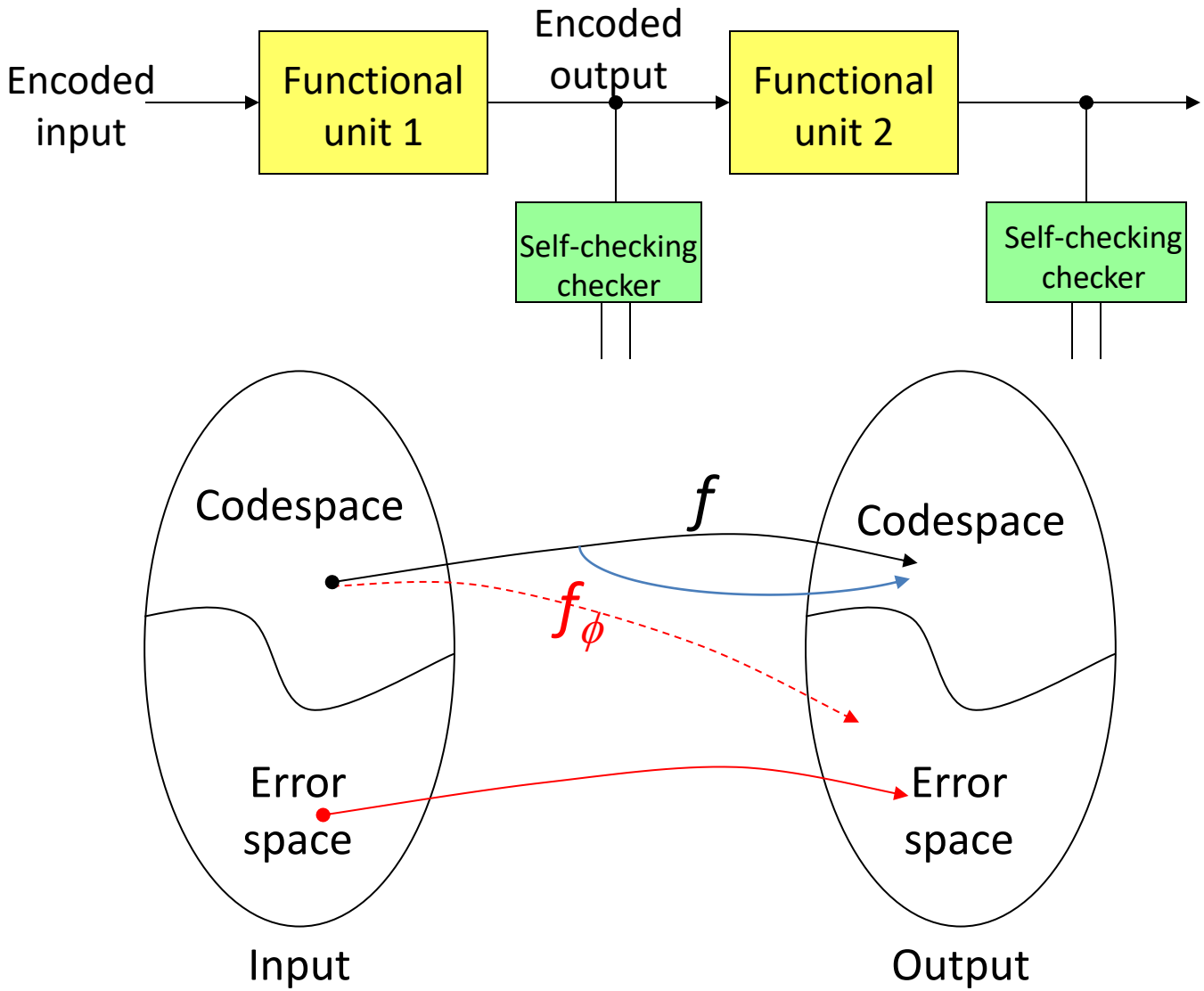


1. Both functional unit and checker okay
2. Only functional unit okay
   – false alarm may be raised, but this is safe
3. Only checker okay
   – we have either no output error or a detectable error
4. Neither functional unit nor checker okay
   – use 2-output checker; a single check signal stuck-at-okay goes undetected, leading to fault accumulation

Code Space and Error Space



Cascade of Self Checking Modules



Definition of Fault Secure and Self Testing

- A circuit L is *Fault Secure (FS)* for an input set N and a fault set F, if for input X in N and for any fault f in F,

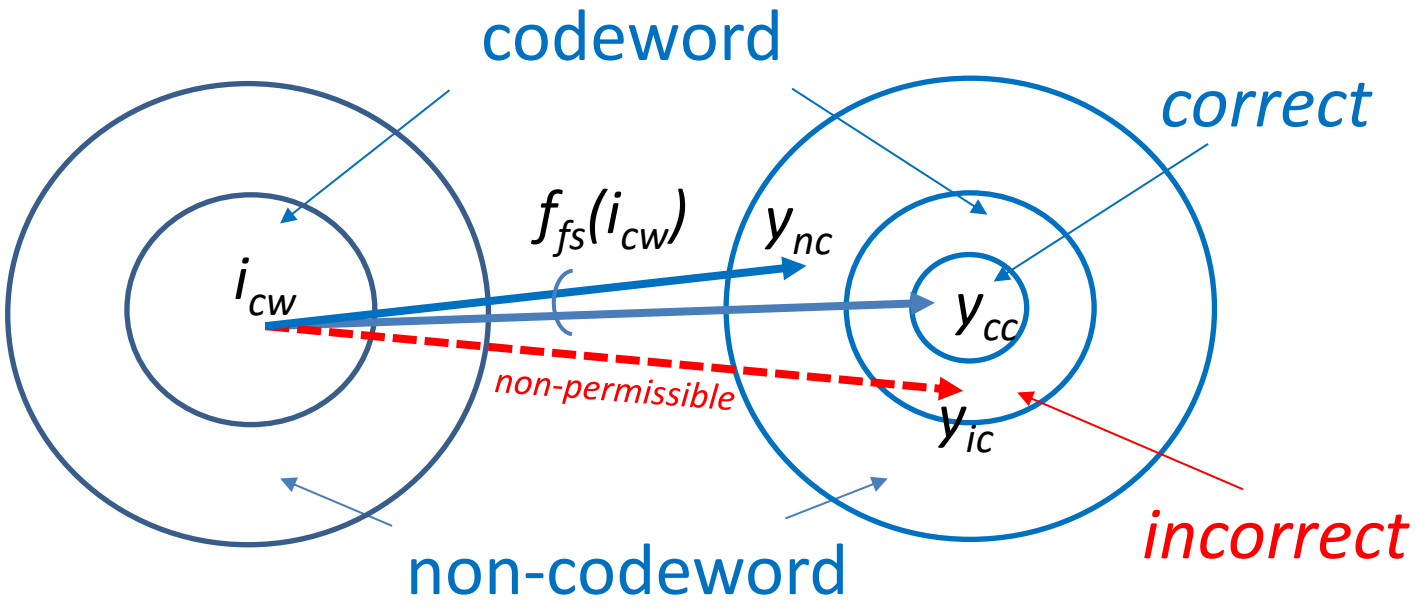
$$Y(X, f) = Y(X, \lambda) \;\; or \;\; Y(X, f) \notin S$$

- A circuit *L* is *Self Testing (ST)* for an input set N and a fault set F, if for every fault f in F there is some input X in N such that $Y(X, f)$ is not in S

S: Output Codespace, λ: Null Fault

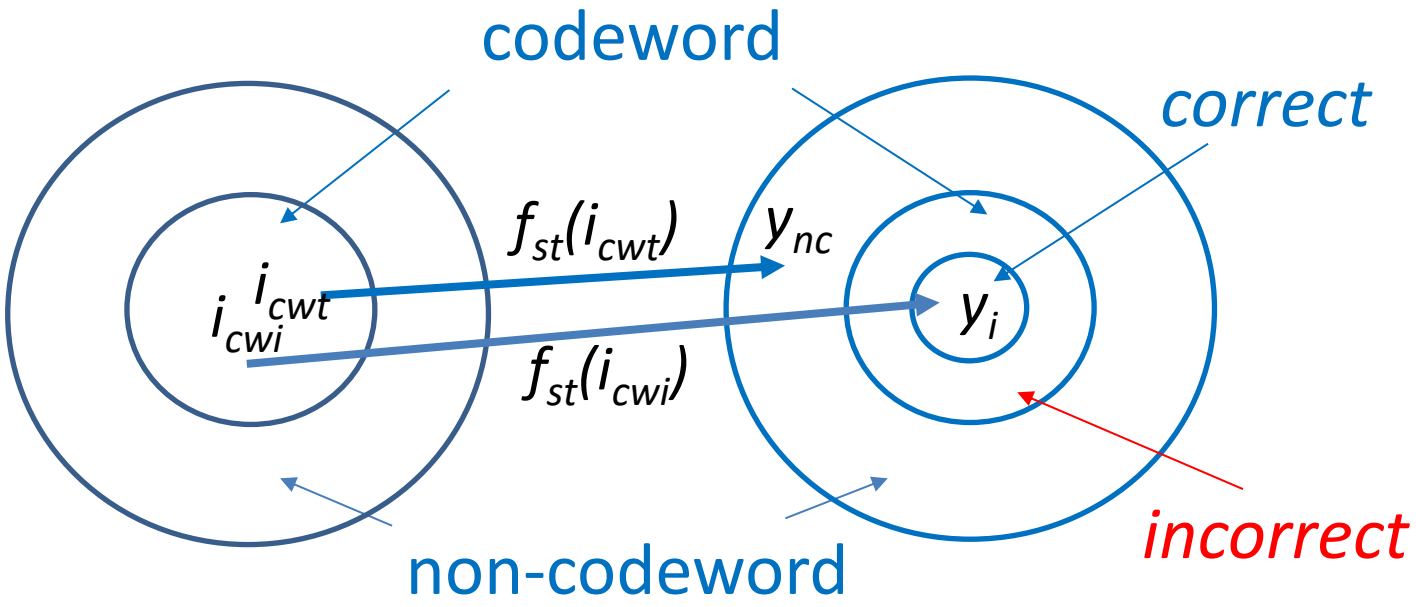
Characteristics of SC circuits

- Fault Secure (FS)
  - The property to ensure that for any possible fault, the circuit output is either a correct codeword or non-codeword



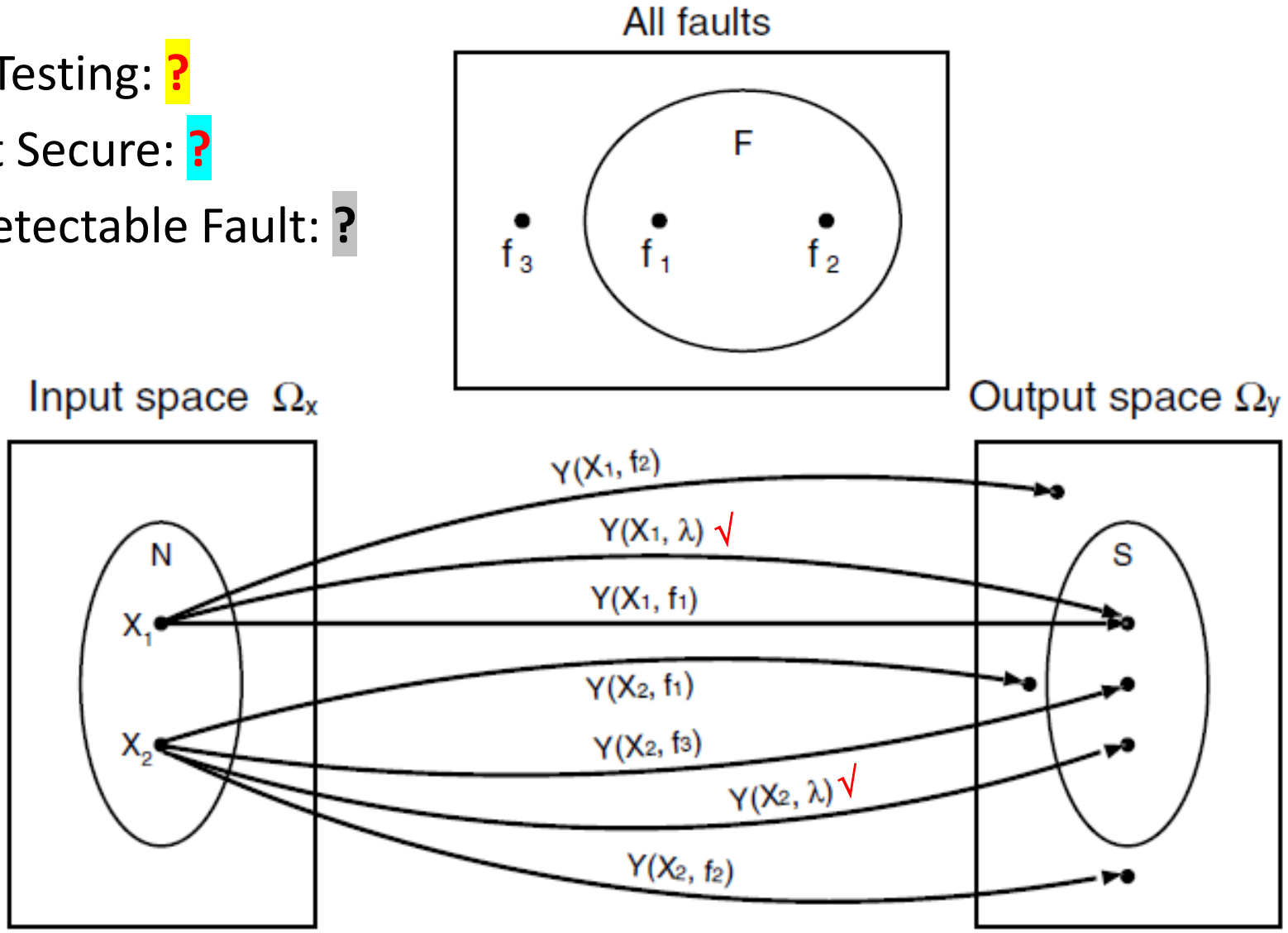
Characteristics of SC circuits

- Self Testing (ST)
  - The property to ensure that for every possible fault, there is at least one input codeword for which the resulting output is a non-codeword



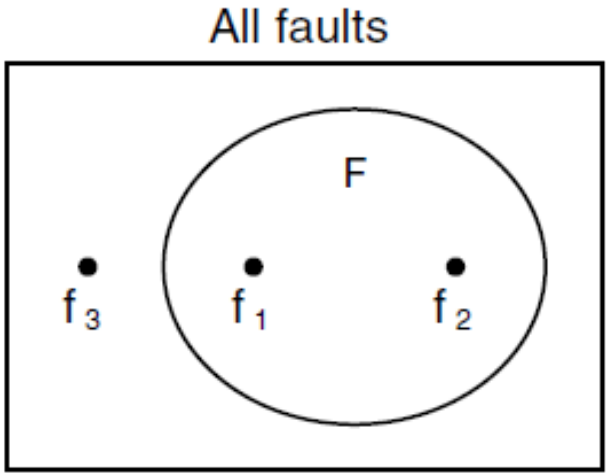
Self Testing & Fault Secure

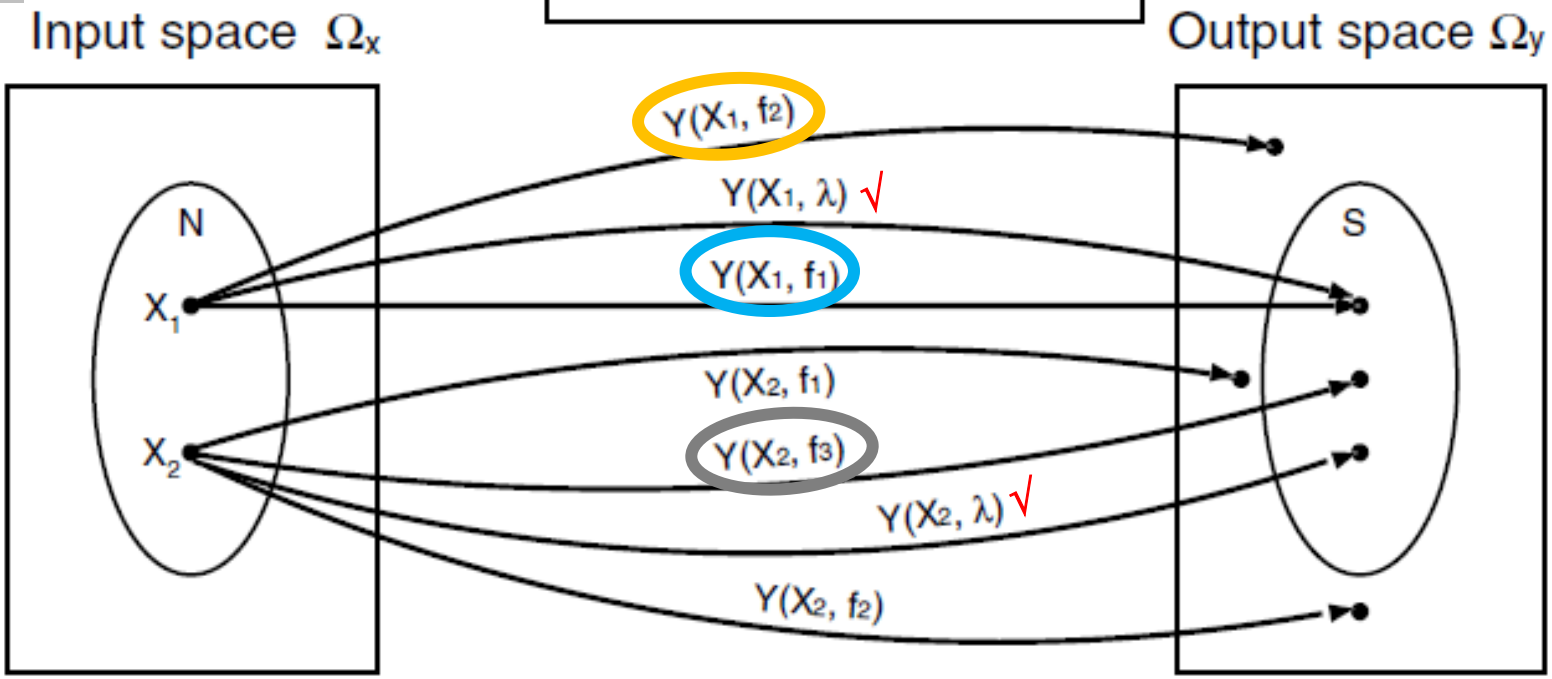
- Self Testing: ?
- Fault Secure: ?
- Undetectable Fault: ?



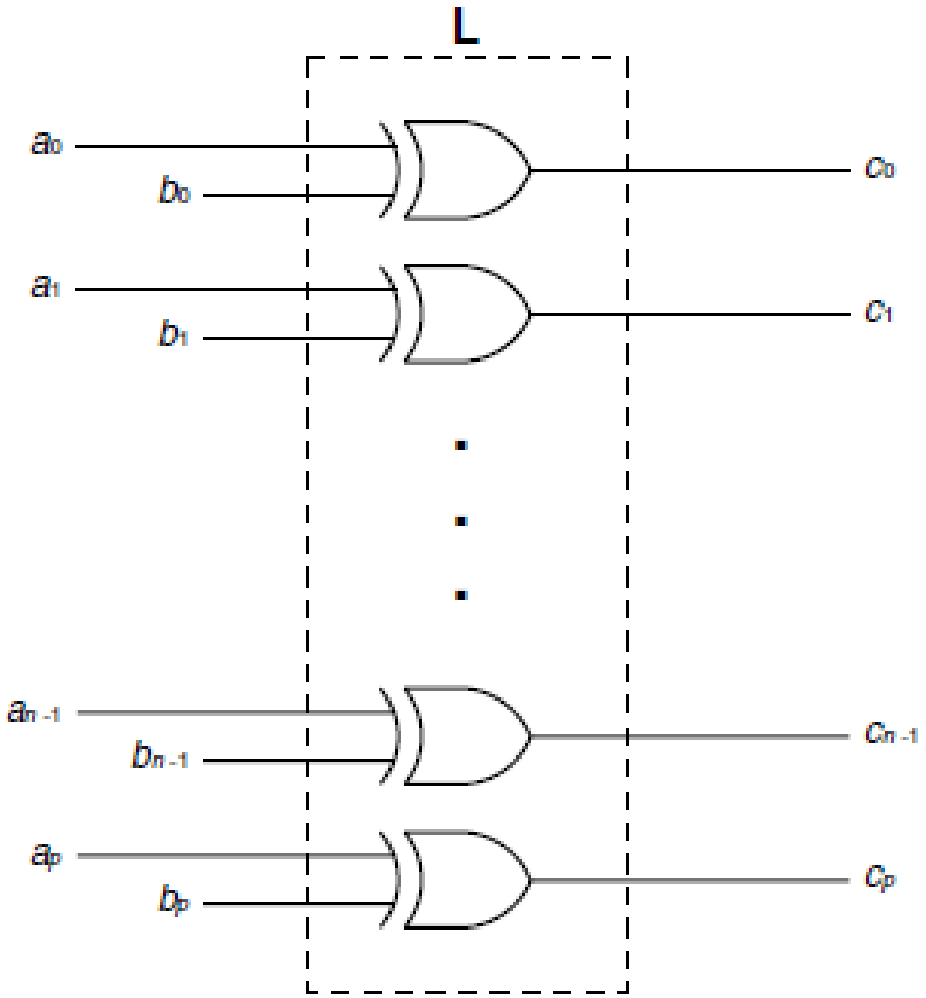
Self Testing & Fault Secure

- Self Testing:
  - **Y(X1,f2)**, Y(X2,f1), Y(X2,f2)
- Fault Secure:
  - **Y(X1,f1)**, Y(X2,f1), Y(X1,f2), Y(X2,f2)
- Undetectable Fault:
  - **Y(X2,f3)**





Self Testing & Fault Secure EXOR circuit

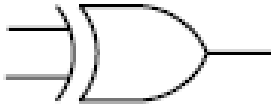


- *Fault Secure*
- *Self Testing*
- *Code Preserving*

$A = (a_0\ a_1\ \ldots\ a_{n-1}\ a_p)$

$B = (b_0\ b_1\ \ldots\ b_{n-1}\ b_p)$

$C = (c_0\ c_1\ \ldots\ c_{n-1}\ c_p)$



: Exclusive-OR (XOR) gate

Totally Self Checking (TSC)

- A circuit is totally self checking if it is both Self Testing and Fault Secure
- An Ideal property but;
  - Not every fault can be detected by codeword
  - Undetectable faults in redundant circuits

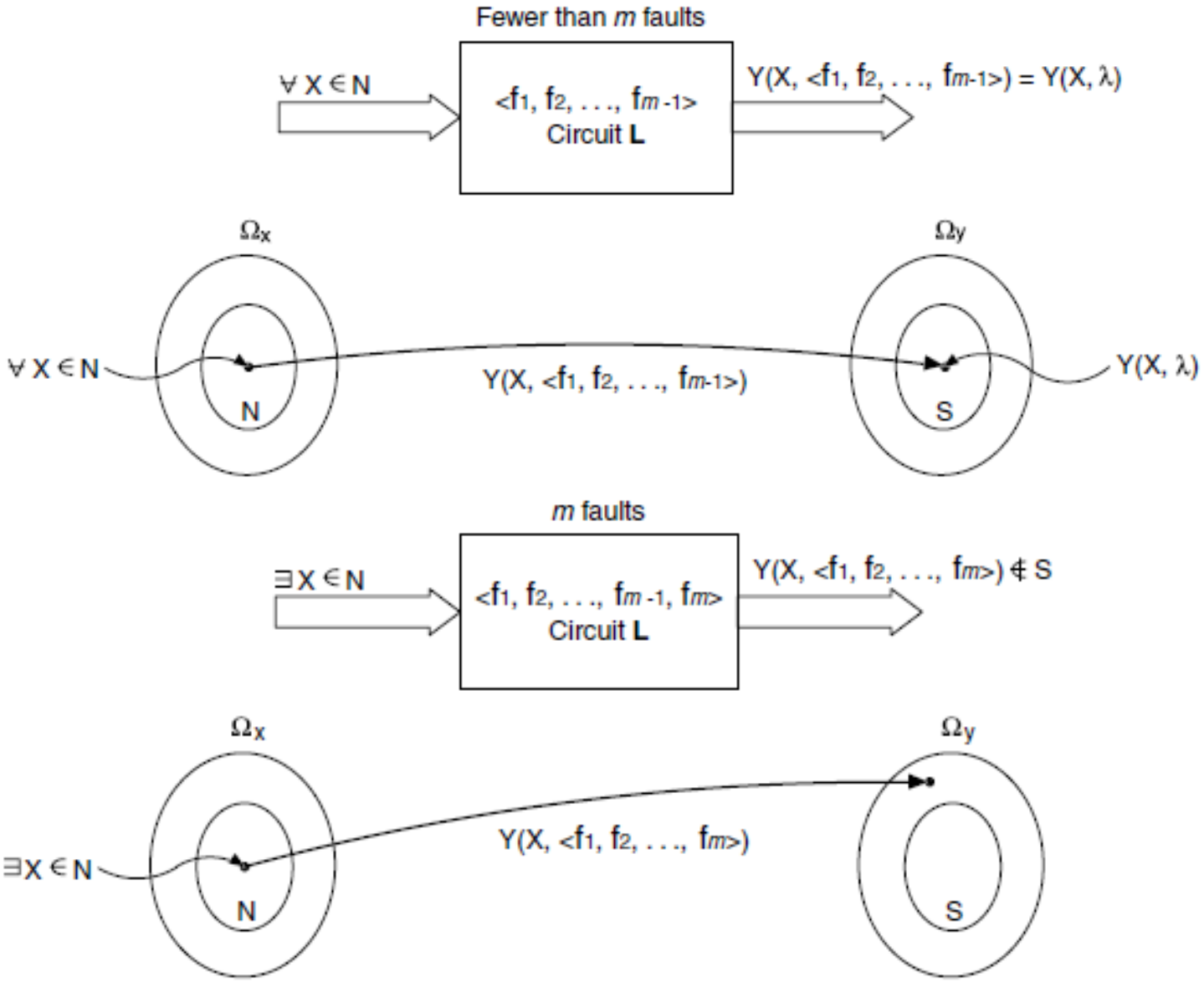
- Even FS but undetectable fault is existing; the next coming faults may invalidate FS property

Strongly Fault Secure (SFS)

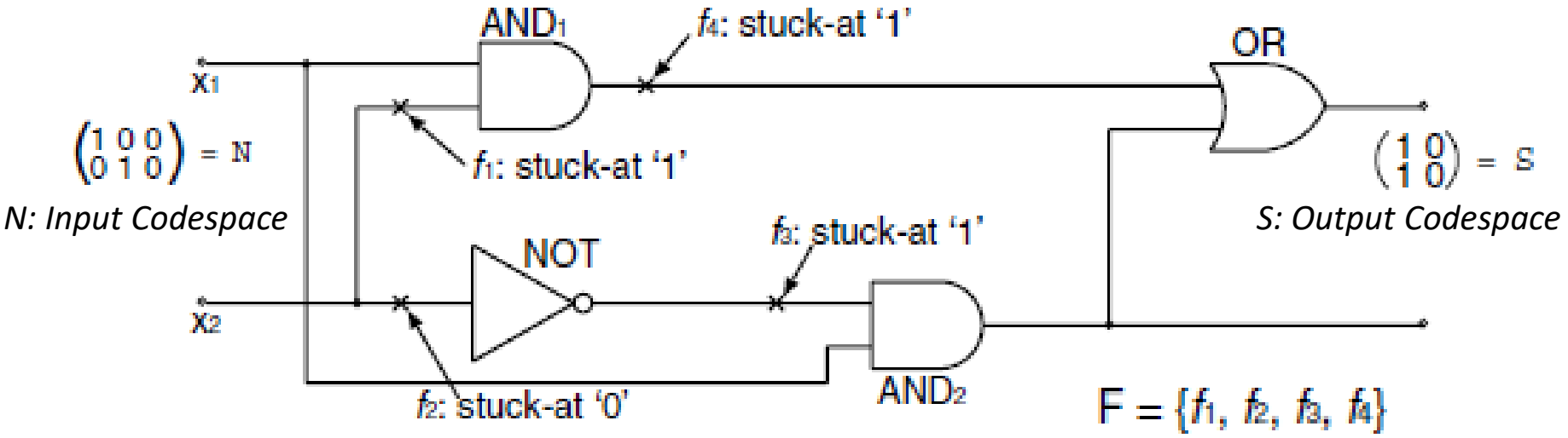
- A circuit is Strongly Fault Secure if 1 or 2;
  1. *TSC* for fault $\{f\}$,
  2. Both *FS* for fault $\{f\}$ AND *SFS* for $F - \{f\}$

- In case the fault $f1 \in \{f\}$ is not be detected, still guarantee that the next fault $f2 \in F - \{f\}$ also satisfy 1 or 2
- The first output error is non-codeword

Strongly Fault Secure *for m faults*



Example of SFS Circuit



- For Input $N = \left\{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right\} = \left\{\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}\right\}$
- **Fault Secure** for the fault sequence $\langle f_1, f_2, f_3 \rangle$

$$Y(X, \langle f_1, f_2, f_3 \rangle) = (Y(X, \lambda) \quad for\ all\ X \in N)$$

- For Input $X_1 \in \left\{\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}\right\}$
- **Strongly Fault Secure** for $\langle f_1, f_2, f_3, f_4 \rangle$

$$Y(X_1, \langle f_1, f_2, f_3, f_4 \rangle) = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \notin S$$

Error Secure

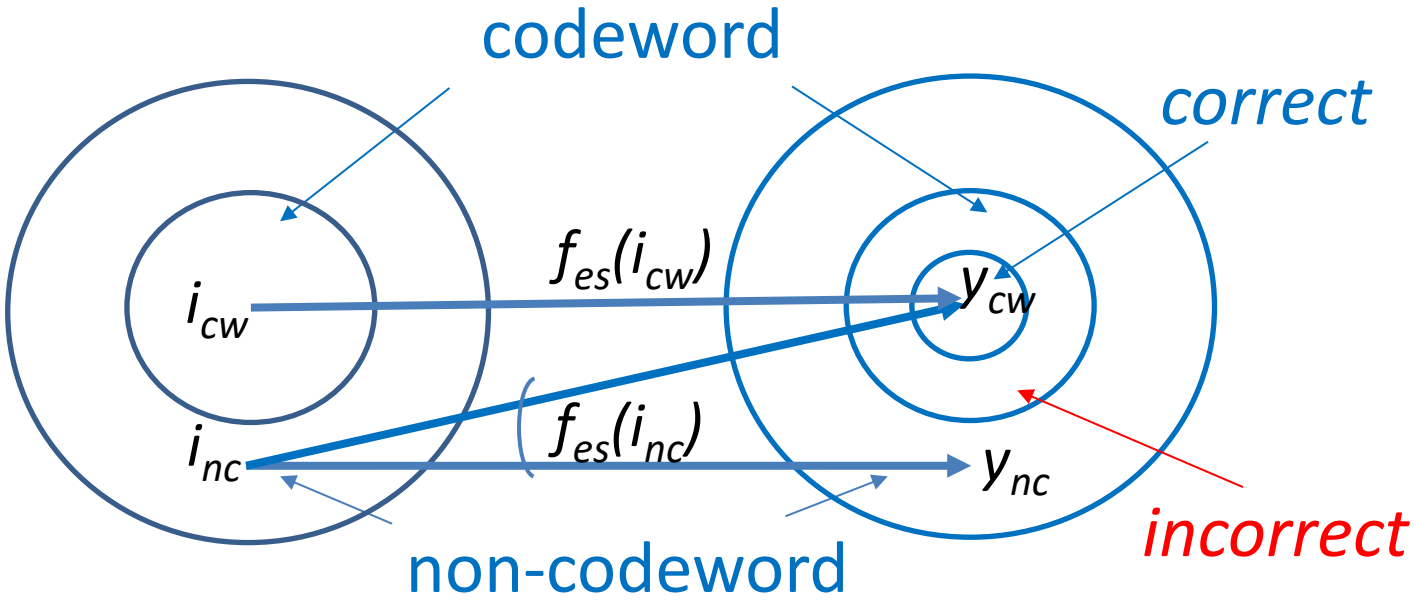
- Considering the behavior of the circuits (self-checking checkers) for non-codeword inputs
- A Circuit L with output codespace S is *Error Secure (ES)* for noncodespace (input) $\Omega_X - N$ if for any input X_q in $\Omega_X - N$, where $X_q = X_p + E, X_p \in N, E \neq 0$

$$Y(X_q, \lambda) \notin S \; or \; Y(X_q, \lambda) = Y(X_p, \lambda)$$

N: Input Codespace, E: Input Error, S: Output Codespace, λ: Null Fault

Error Secure Circuits

- Error Secure (ES)
  - The property to ensure that for any non-codeword input, the circuit output will be either a correct codeword (Fault Masking) or non-codeword



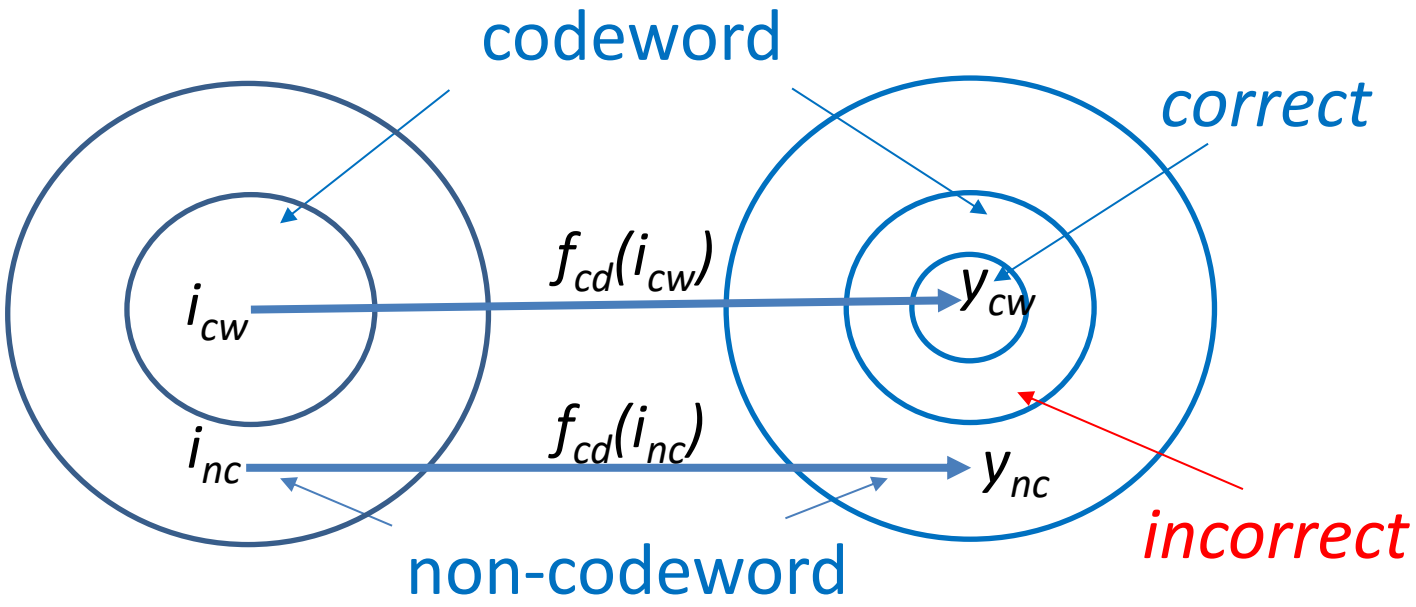
Error Preserving/Code Disjoint

- Synonymous with Code Disjoint (CD)
- A Circuit L with output codespaces S is Error Preserving for noncodespaces $\Omega_X - N$, if for any input X in $\Omega_X - N$

$$Y(X, \lambda) \notin S$$

Code Disjoint Circuits

- Code Disjoint (CD)
  - The property of fault-free circuits to ensure that for any codeword/ non-codeword input, the output will be a correct codeword/ non-codeword



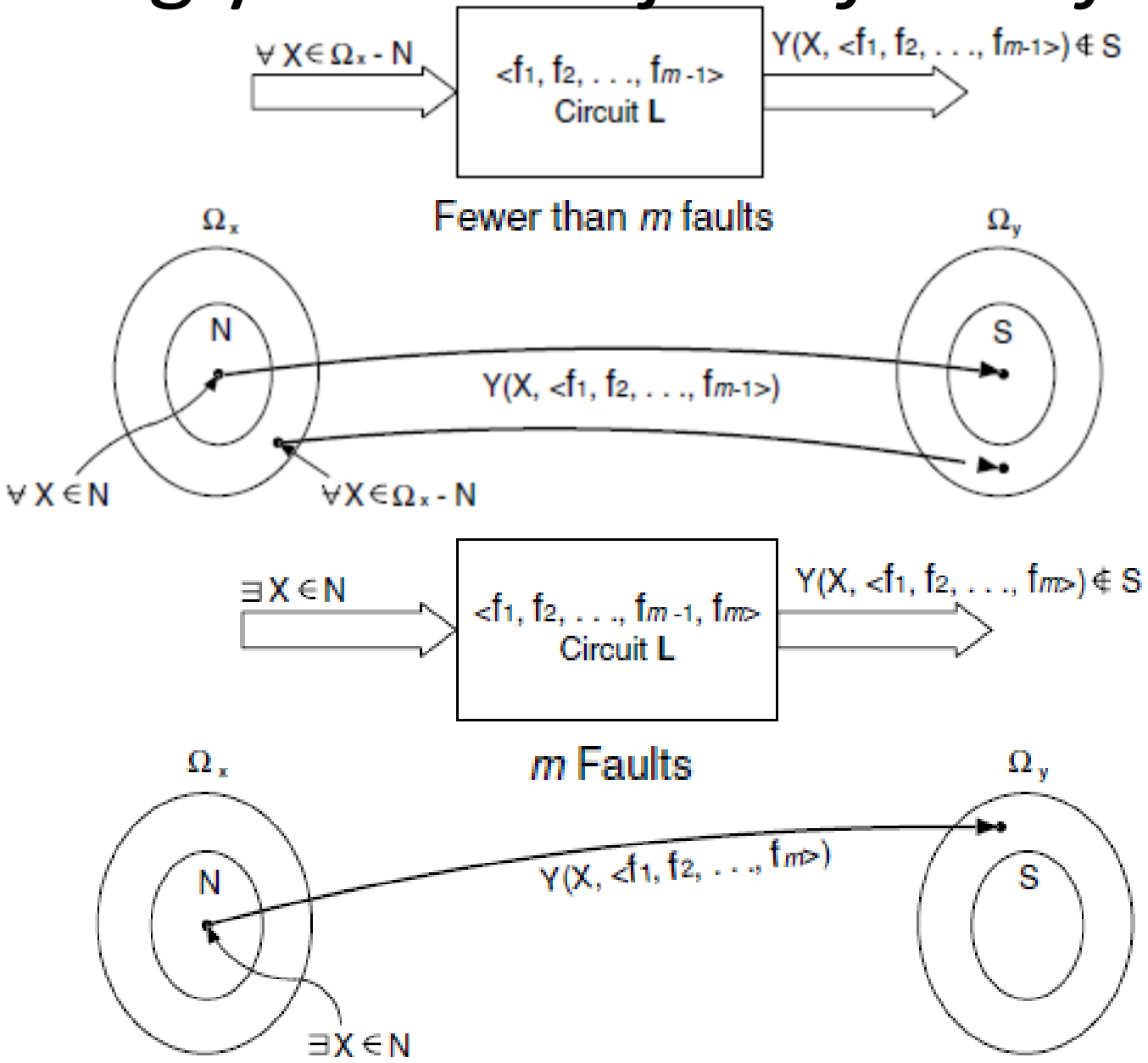
Strongly Code Disjoint

- A circuit is Strongly Code Disjoint (SCD),

  if 1 or 2

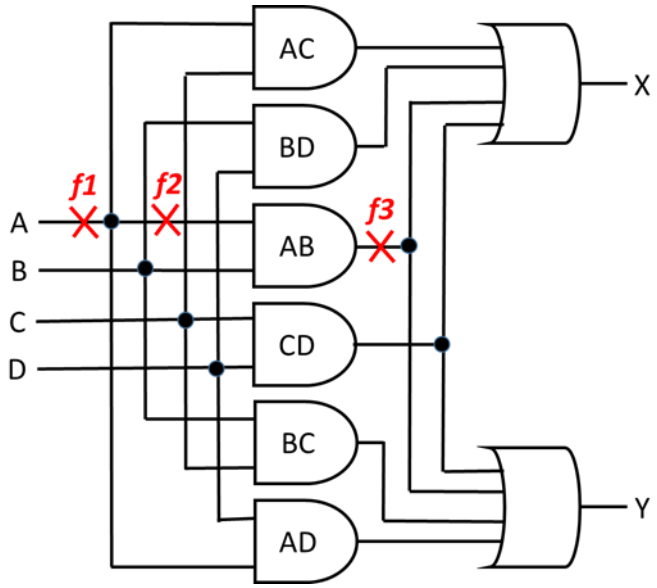
  1. CD before the fault f occur; and ST for fault $\{f\}$
  2. CD before the fault f occur; and SCD for $F - \{f\}$ when the fault f occur

Strongly Code Disjoint *for m faults*



$$\text{codeword}\{(A,B),(C,D),(X,Y)\} \in \{(0,0),(0,1),(1,0)\}$$

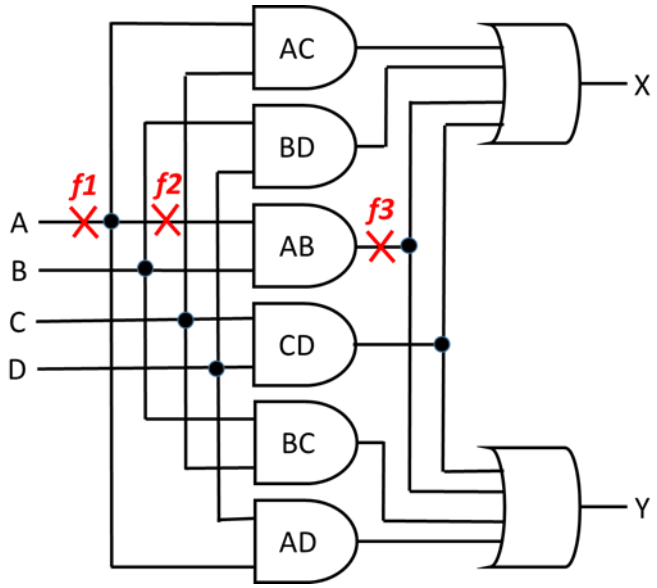
1. วงจรนี้เป็น CD? ในกรณี Fault-free
2. วงจรนี้เป็น FS? ST? สำหรับ {f1} ในกรณีเกิด Stuck-at-0 Fault
3. วงจรนี้เป็น FS? ST? สำหรับ {f2} ในกรณีเกิด Stuck-at-0 Fault
4. วงจรนี้เป็น FS? ST? สำหรับ {f3} ในกรณีเกิด Stuck-at-1 Fault



AB	CD	XY			
		Normal	f1sa0	f2sa0	f3sa1
00	00	00			
00	01	00			
00	10	00			
01	00	00			
01	01	10			
01	10	01			
10	00	00			
10	01	01			
10	10	10			
00	11	11			
01	11	11			
10	11	11			
11	00	11			
11	01	11			
11	10	11			
11	11	11			

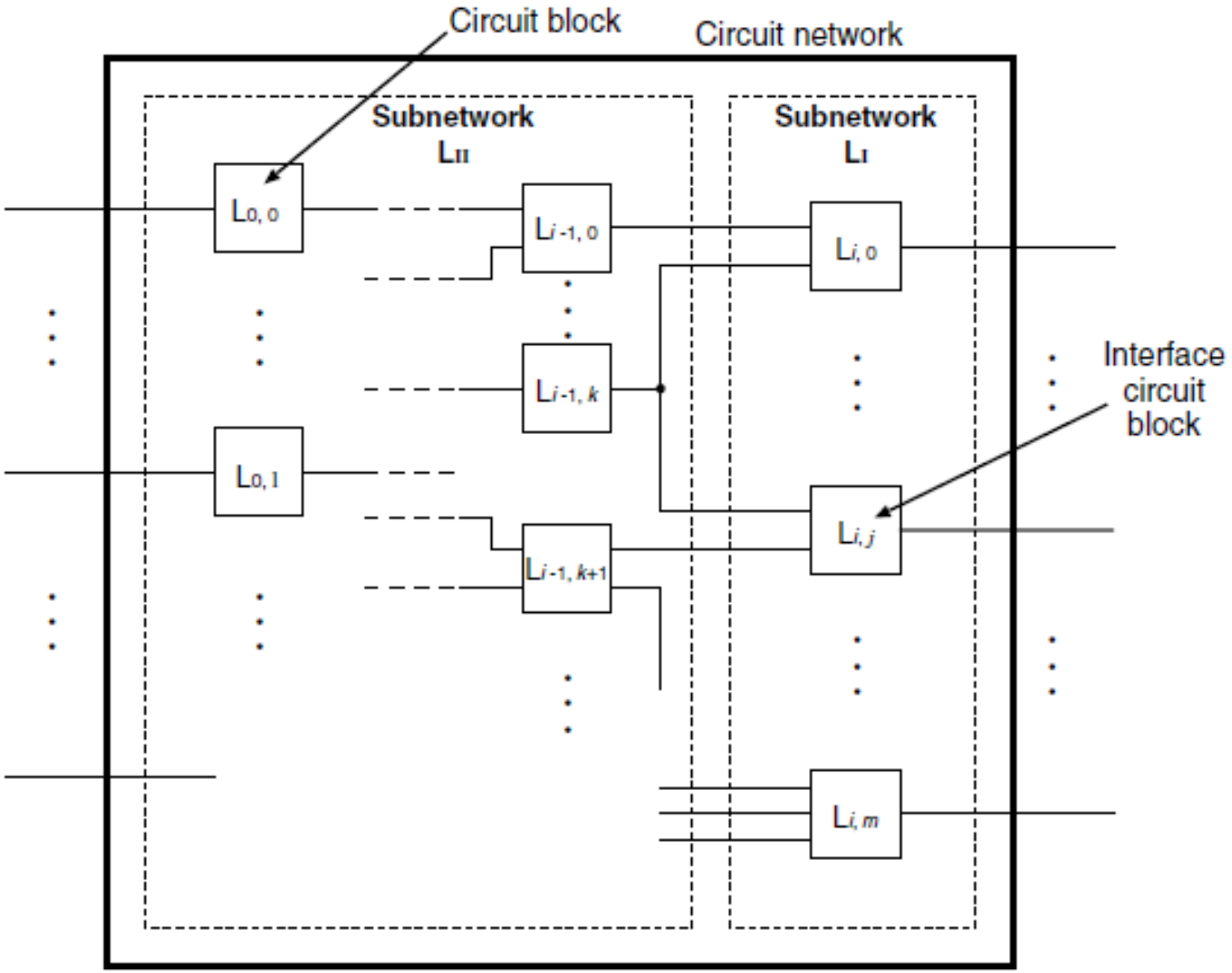
$$\text{codeword}\{(A,B),(C,D),(X,Y)\} \in \{(0,0),(0,1),(1,0)\}$$

1. วงจรนี้เป็น CD? ในกรณี Fault-free
2. วงจรนี้เป็น FS? ST? สำหรับ {f1} ในกรณีเกิด Stuck-at-0 Fault
3. วงจรนี้เป็น FS? ST? สำหรับ {f2} ในกรณีเกิด Stuck-at-0 Fault
4. วงจรนี้เป็น FS? ST? สำหรับ {f3} ในกรณีเกิด Stuck-at-1 Fault



AB	CD	XY			
		Normal	f1sa0	f2sa0	f3sa1
00	00	00	00	00	11
00	01	00	00	00	11
00	10	00	00	00	11
01	00	00	00	00	11
01	01	10	10	10	11
01	10	01	01	01	11
10	00	00	00	00	11
10	01	01	00	01	11
10	10	10	00	10	11
00	11	11	11	11	11
01	11	11	11	11	11
10	11	11	11	11	11
11	00	11	00	00	11
11	01	11	10	11	11
11	10	11	01	11	11
11	11	11	11	11	11

Self Checking Logic Network



Theorem & Corollary

Theorem:

"If the subnetwork LI consisting of the output interface circuit blocks is *SFS* and *SCD* for fault set *FI*, and if the remaining subnetwork *LII* consisting of the internal circuit blocks is *SFS* for fault set *FII*, then the total network is *SFS* for all fault sequences in *FI* and *FII*"

Corollary:

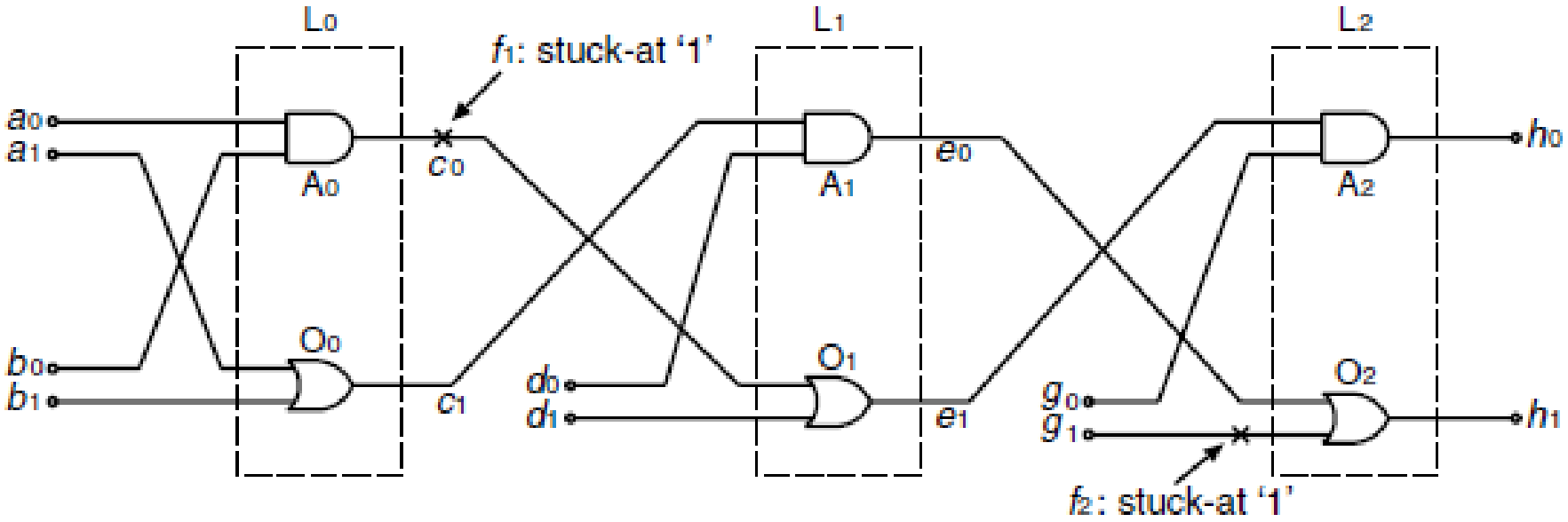
"If the subnetwork LI consisting of the output interface circuit blocks is *SFS* and *CD*, and if the remaining subnetwork *LII* consisting of the internal circuit blocks is *SFS*, then the total network is *SFS* for all fault sequences in either *LI* or *LII*, but not in both"

Inverter-free Network

Theorem:

- "A network that consists of only *Inverter-free* combinational circuit blocks and in which the block input/output interfaces are encoded with *unidirectional error detecting codes* (eg., the m-out-of n code and the Berger code) is *SFS* with respect to unidirectional faults"

Cascaded Inverter-free Network



- SFS for the unidirectional fault sequence $< f1, f2 >$ and for restricted inputs
- Each circuit block is not always CD
- Non-codewords can appear at embedded interfaces of the network before a *non-codeword* is produced as the first erroneous output

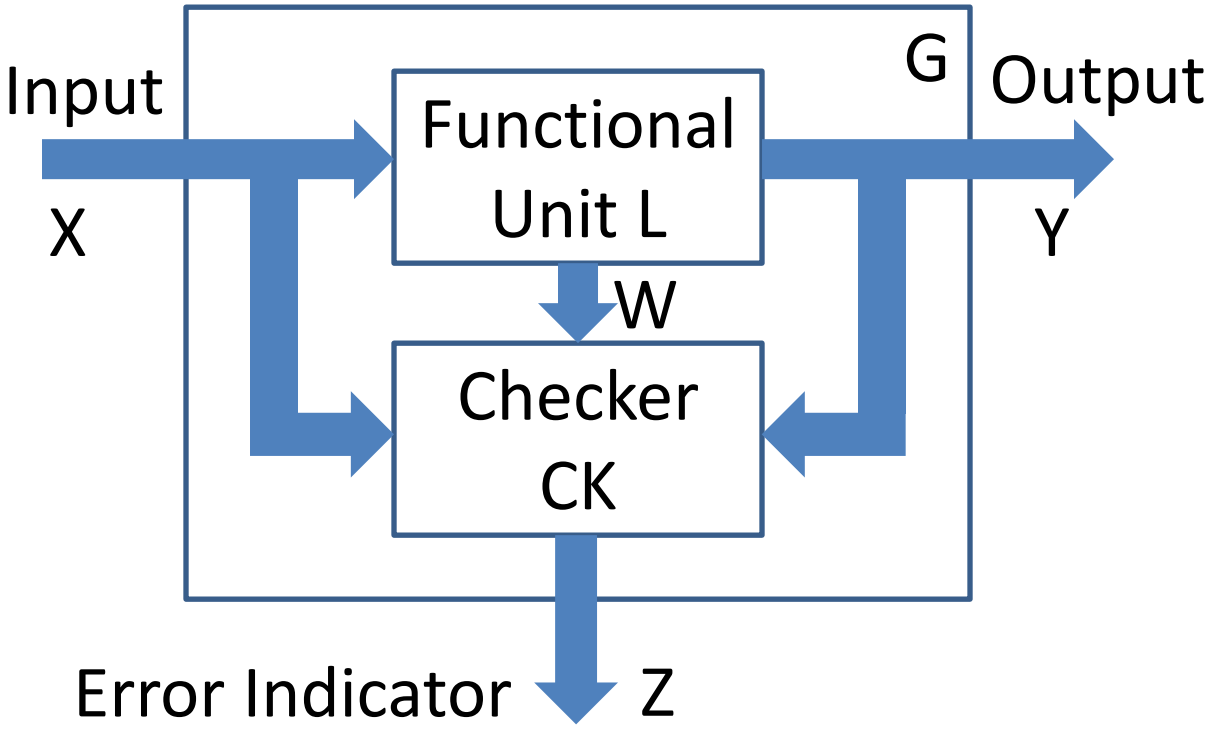
Error Secure and Error Propagating

- A circuit block interface is *Error Secure* (ES)* at a given fault stage if any error that can occur at the interface *never causes an incorrect codeword* at any observable interface during the fault stage
- A circuit block interface is *Error Propagating (EP)* at a given fault stage if any error that can occur at the interface *eventually* produces a *non-codeword* at some observable interface during the fault stage

- Former ES: Block interfaces that may include some faults within them
- Latter ES*: Fault-free circuit blocks

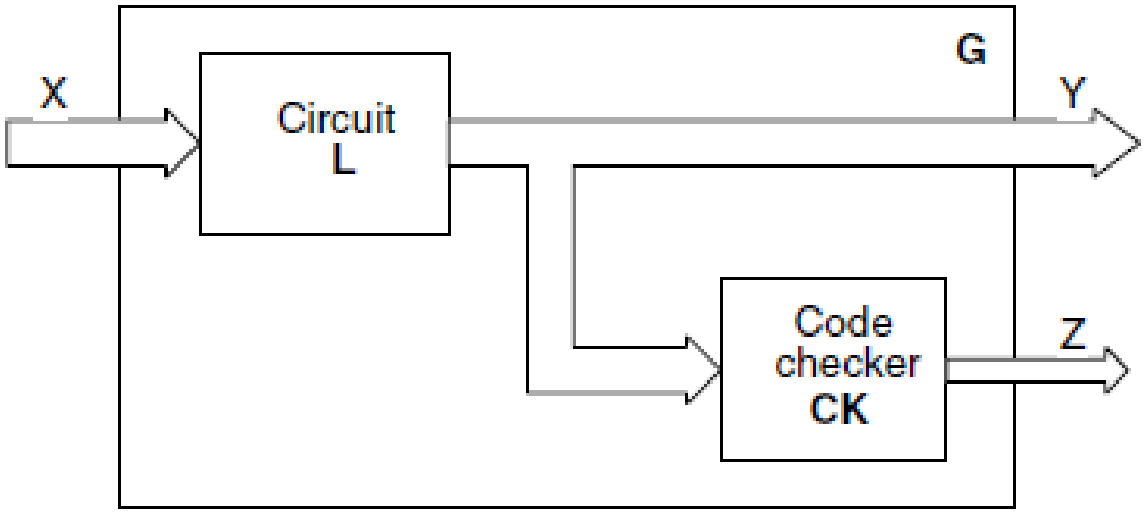
Checkers

Self Checking Model



Types of Checker

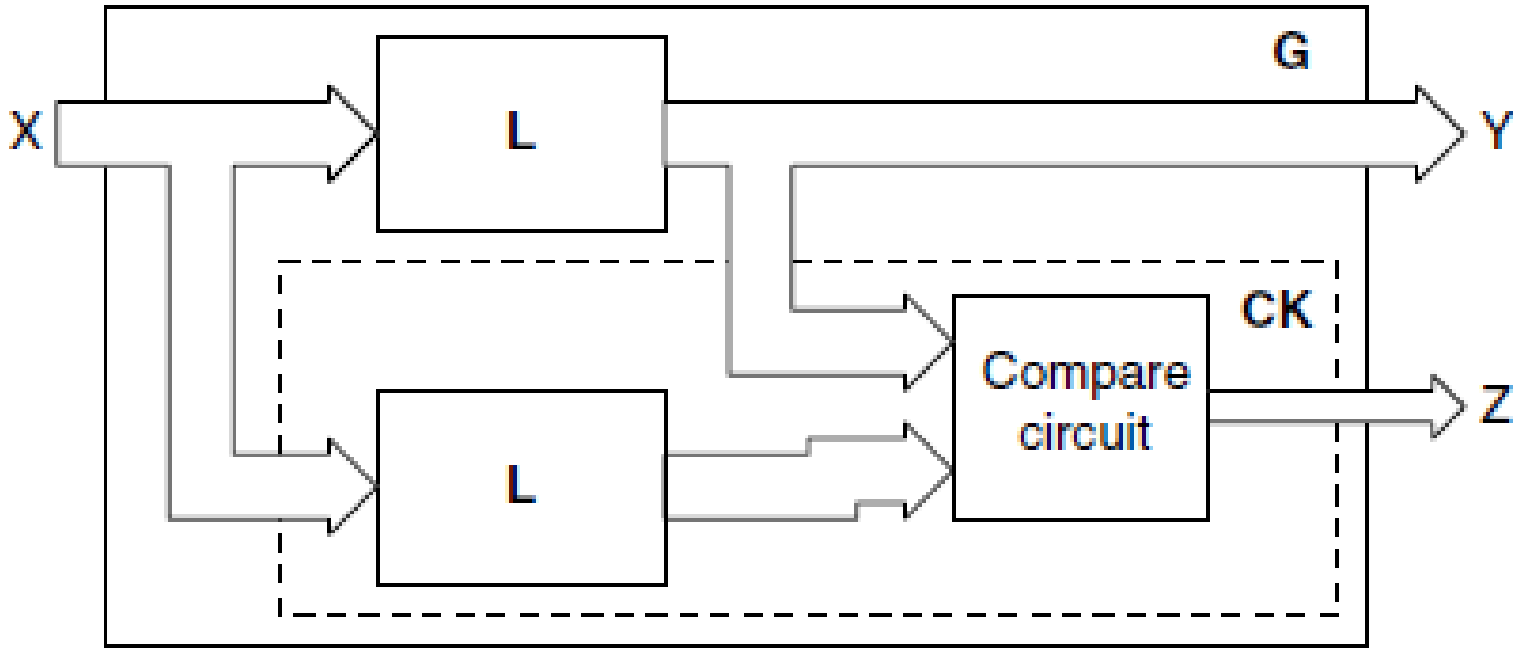
- Code Checker/ Output encoded Checker



- Prediction Checker/ Output Non-encoded Checker
  - Duplication checker, Parity prediction checker, Input regeneration checker

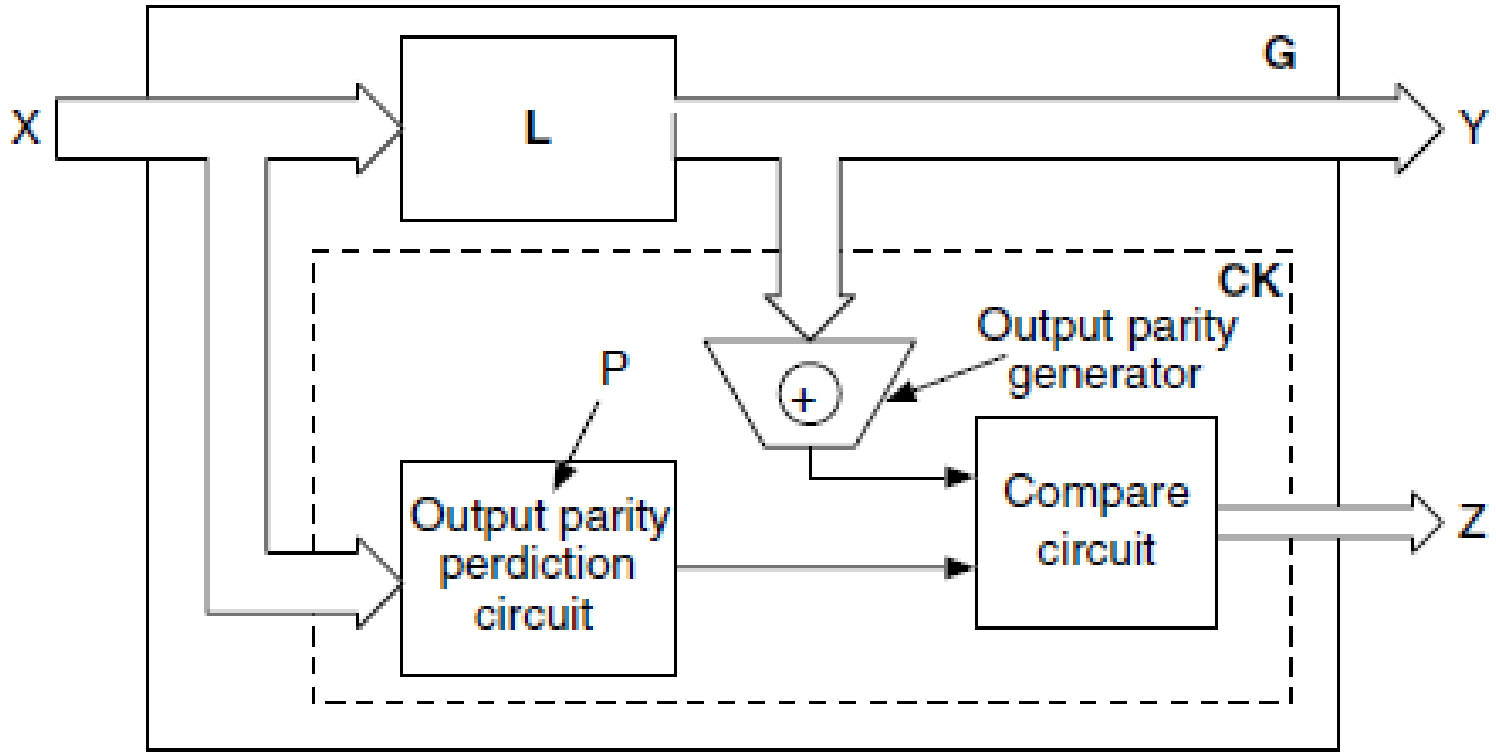
Duplication Checkers

- The functional circuit L is duplicated, and the outputs are compared



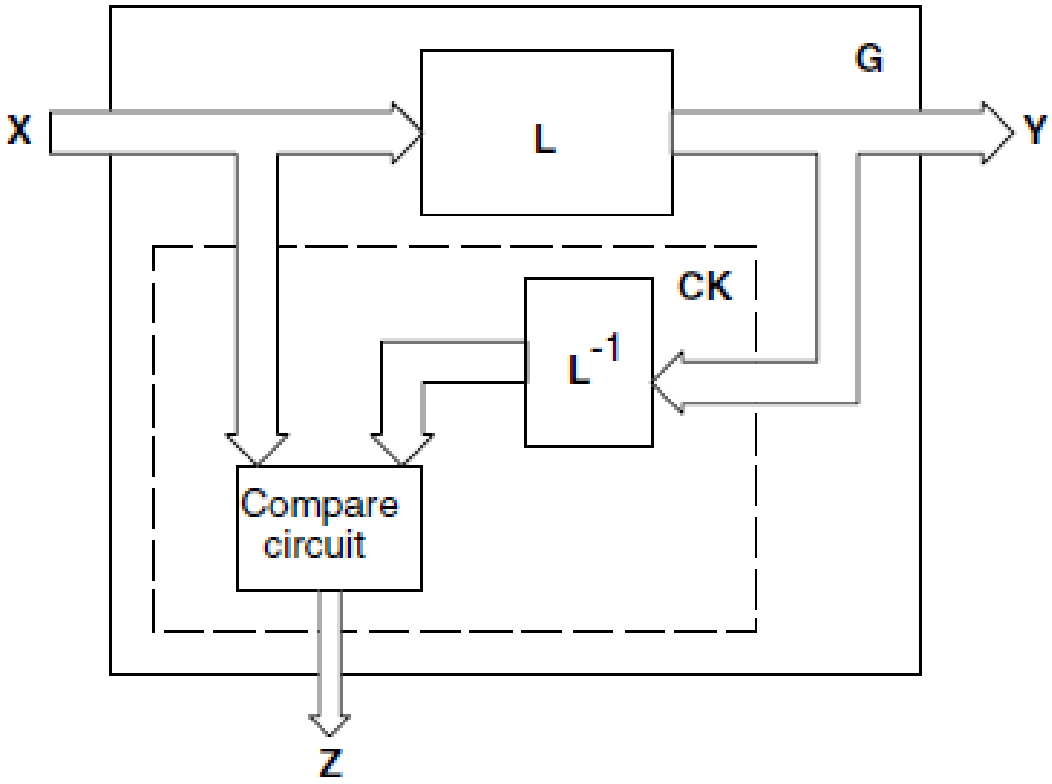
Parity Prediction Checkers

- Output parity prediction P from input X
- Output parity generator from output Y
- Compare both output



Input Regeneration Checkers

- Regenerate inputs from outputs by inverse logic
- Then compare wt the original inputs

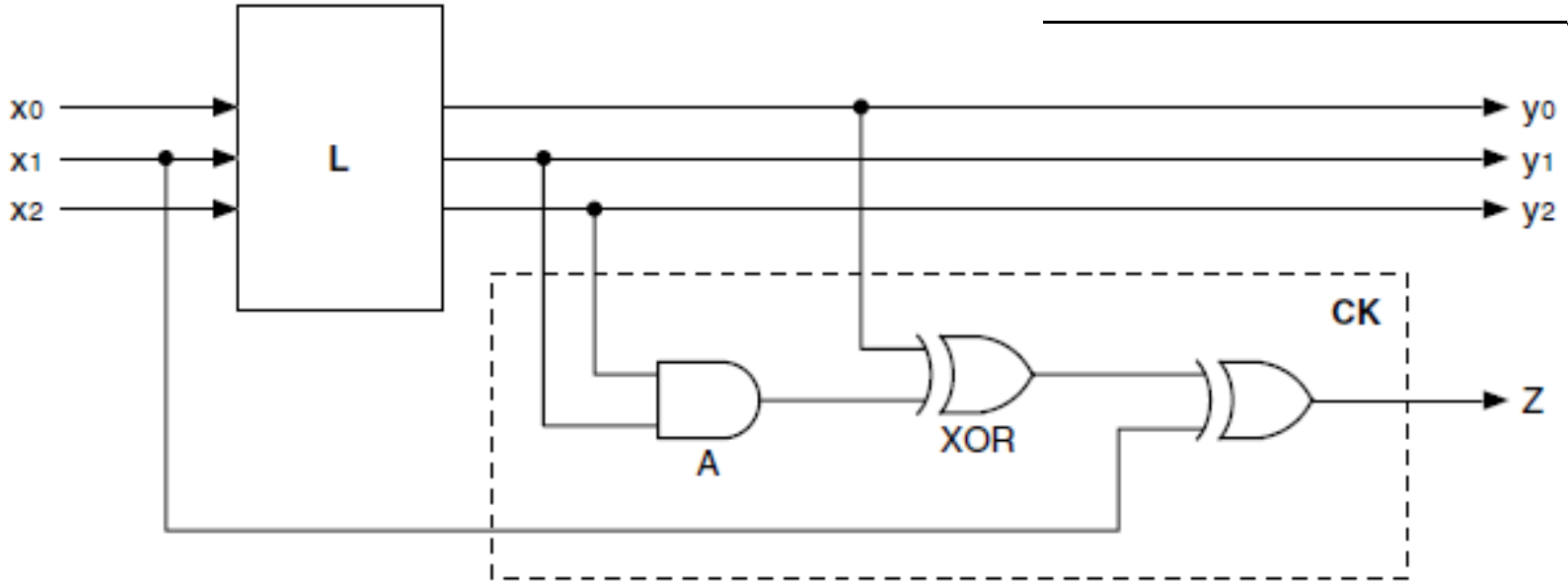


Simplified Regeneration Checkers

- Select only 1 input to check

$$x_1 = y_0 \oplus y_1 \cdot y_2$$

x_0	x_1	x_2	y_0	y_1	y_2
0	0	0	1	1	1
0	0	1	0	0	1
0	1	0	1	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	1	0
1	1	0	1	1	0
1	1	1	1	0	1



Self Checking Checker

Lemmas:

1. For single faults it is necessary that any checker must have at least two outputs and that no outputs take constant values
2. The Fault Secure property is not necessary for a checker

Self Checking Checker Properties

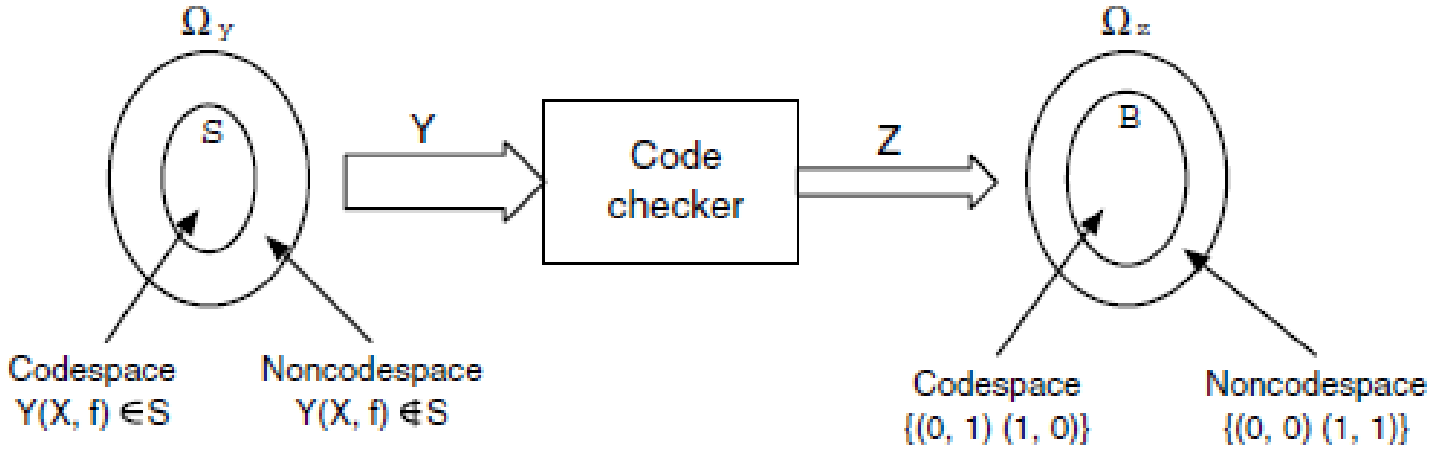
- 2 output lines
- Code-Disjoint (CD)
  - Codeword inputs generate only codeword outputs
  - Non-codeword inputs generate only non-codeword outputs
- Self Testing



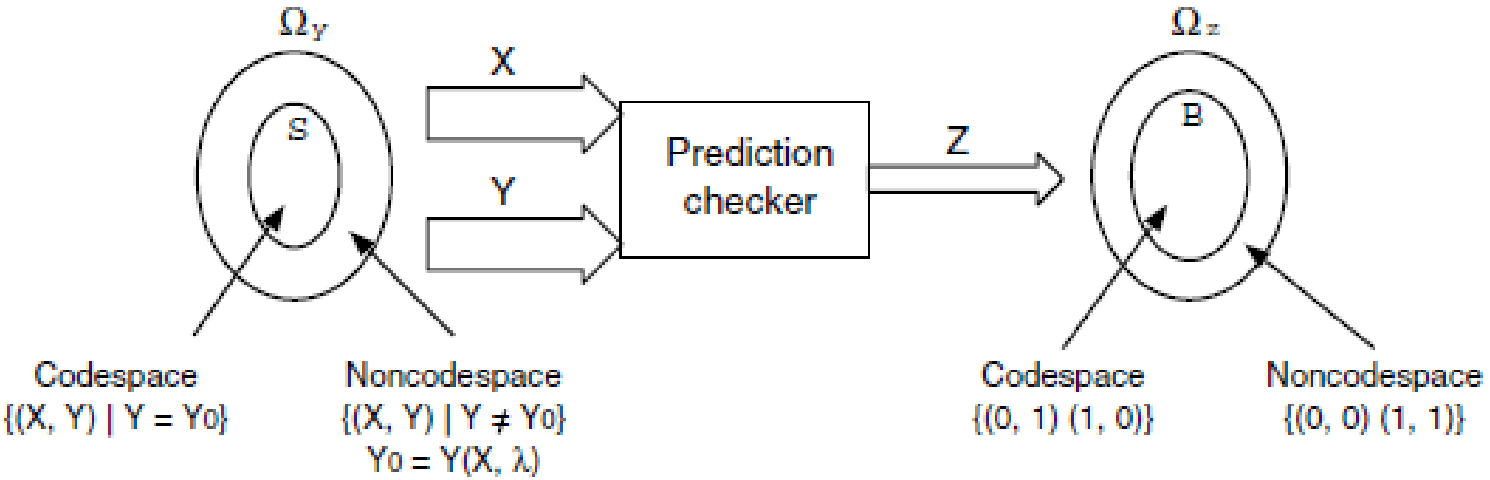
- Strongly Code-Disjoint (SCD) if 1 or 2
  1. CD before the fault f occur; and ST for fault $\{f\}$
  2. CD before the fault f occur; and SCD for $F - \{f\}$ when the fault f occur

Input and Output Space of Checkers

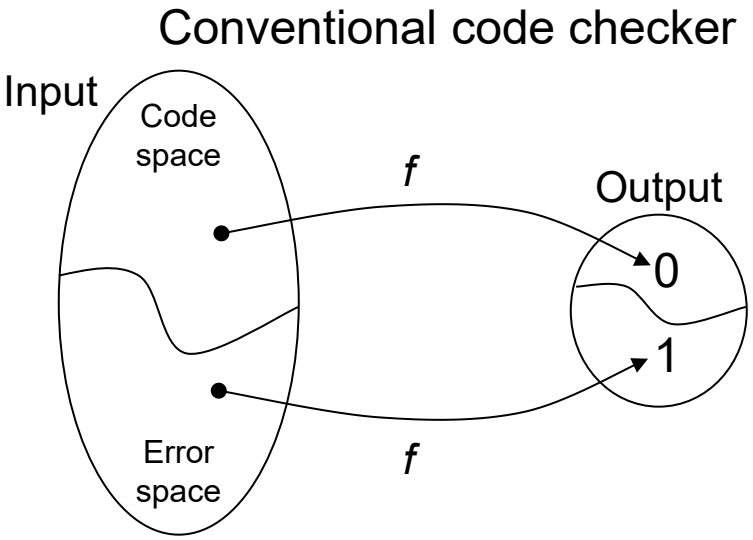
- Code Checker



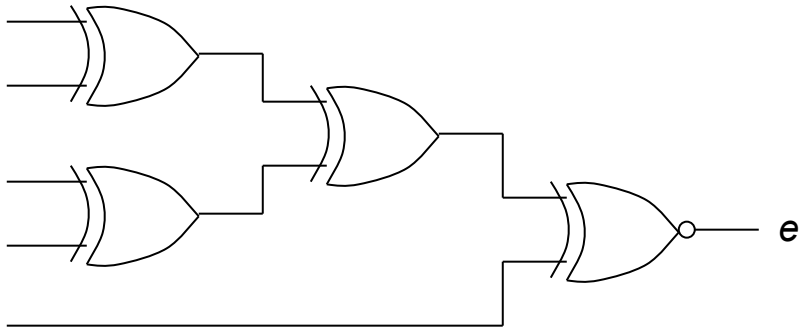
- Prediction Checker



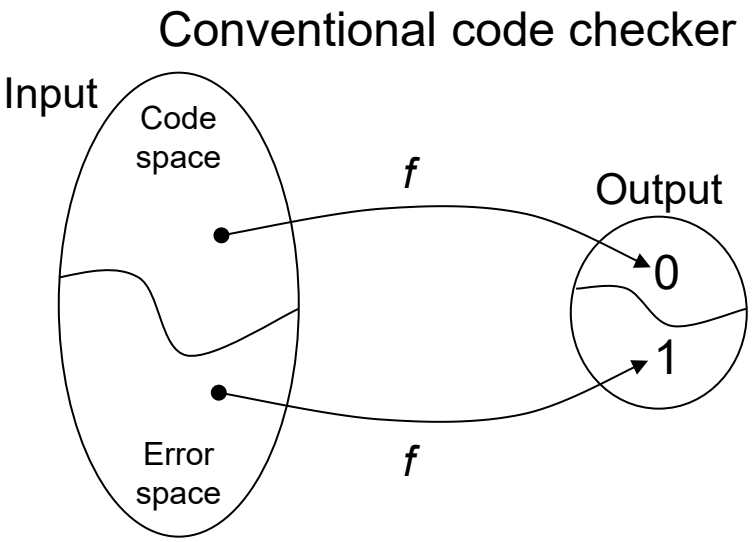
What's wrong?



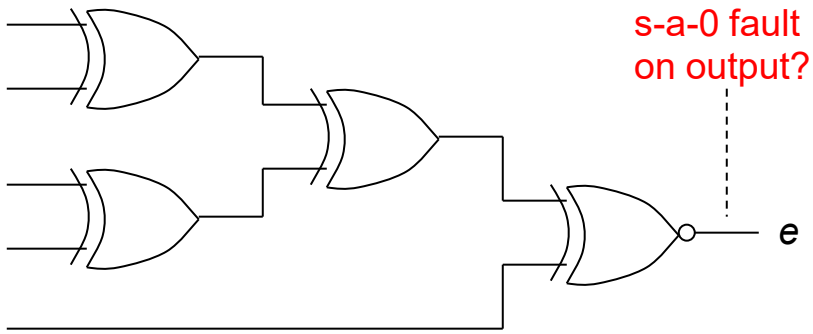
Example: 5-input odd-parity checker



Non-Self Testing Parity Code Checker

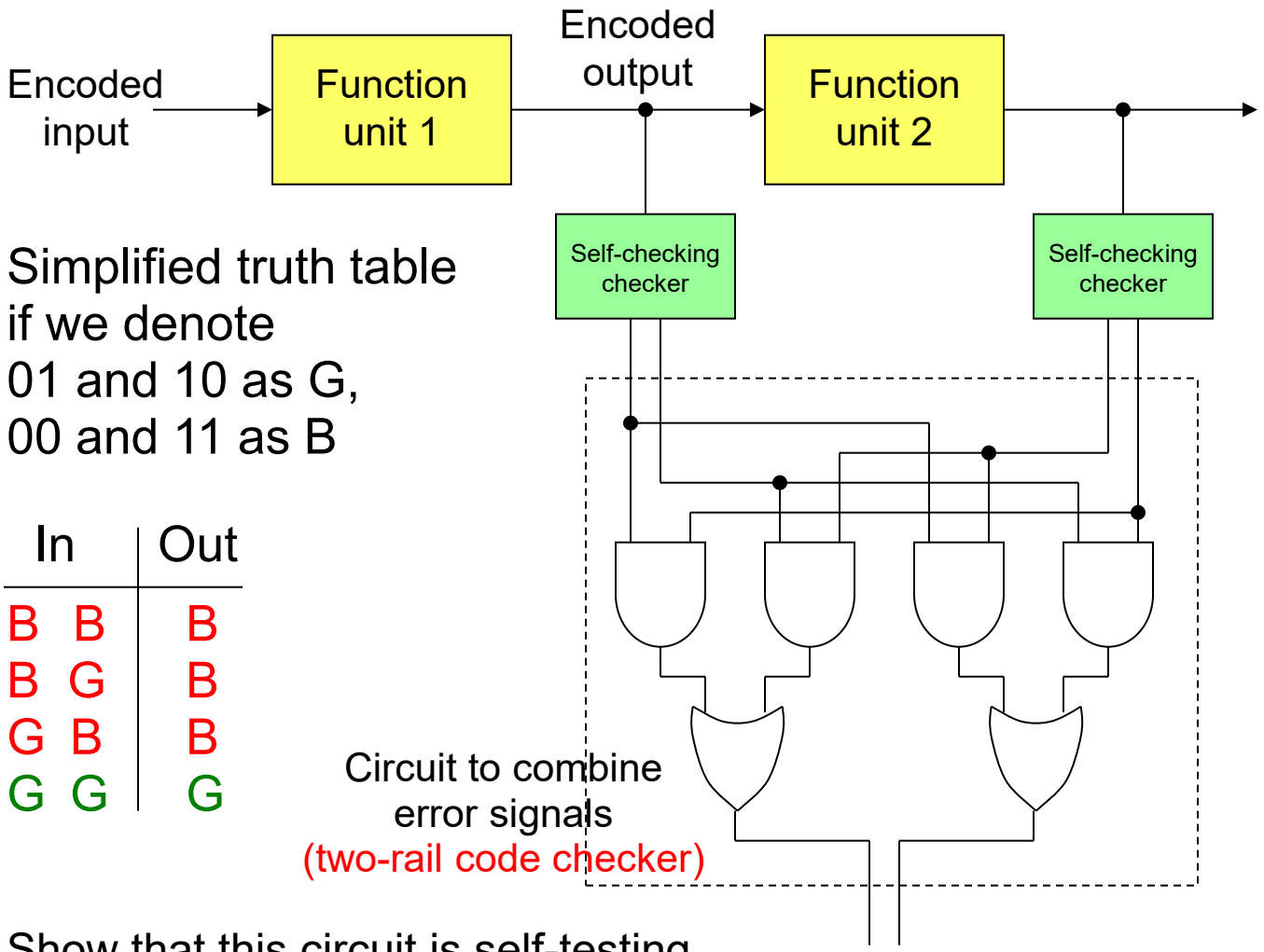


Example: 5-input odd-parity checker



- Single output line
- Code-Disjoint
- non-Self Testing

Totally Self Checking Error Signal Combining



Simplified truth table
if we denote
01 and 10 as G,
00 and 11 as B

In		Out
B	B	B
B	G	B
G	B	B
G	G	G

In				Out	
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	0
0	1	0	0	0	0
0	1	0	1	0	1
0	1	1	0	1	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	1	0
1	0	1	0	0	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

Show that this circuit is self-testing

Implementations of Checker

- Parity Code Checker
- Two Rail Code Checker
- 1-of-n Code Checker
- m-of-2m Code Checker
- TSC Checker for *k*-out-of-*n* Code
- TSC Checker for Separable Codes
- Generalized Prediction Checker (GPC)

Parity Code Checker

- Using Exclusive-OR gates in a Tree Structure
- Dividing inputs into Two Parity-Tree

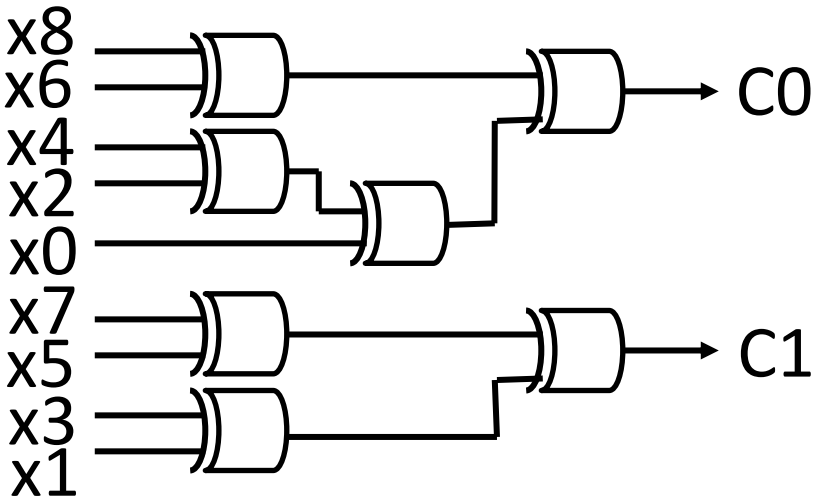
Theorem:

- The parity-tree circuit that is divided into two independent parity-tree subcircuits is a self-testing checker for all odd-parity encoded inputs and for all single faults

Self Testing Parity Code Checker

- 2 output lines
- (C1,C0) = {(0,1),(1,0)} = codeword
- (C1,C0) = {(0,0),(1,1)} = non-codeword

Ex. 8 bit data bit + 1 bit odd-parity bit



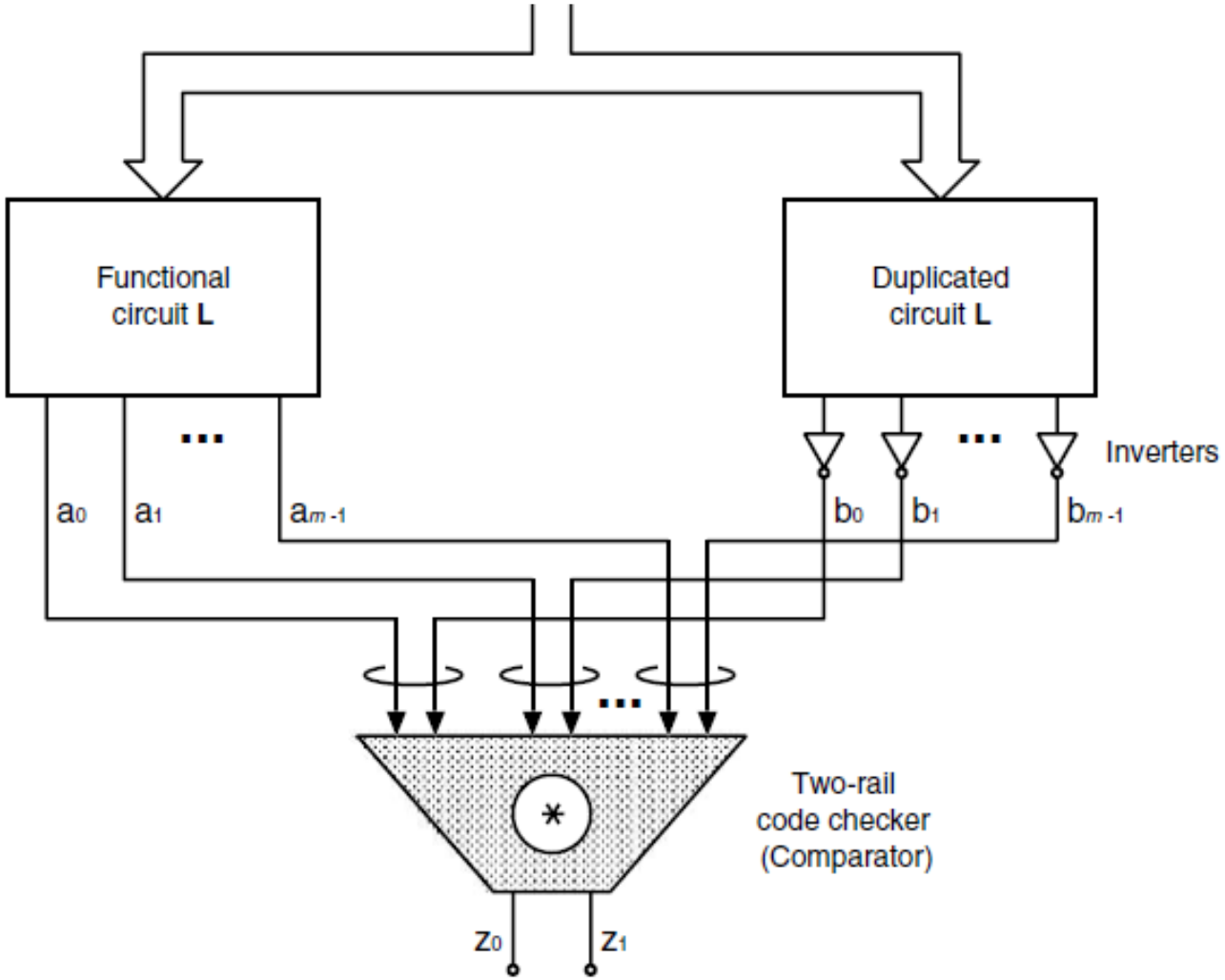
Self Testing Checker

- 2 output lines
- Code-Disjoint
- Self Testing

(number of 1s is even)

Two Rail Code Checker

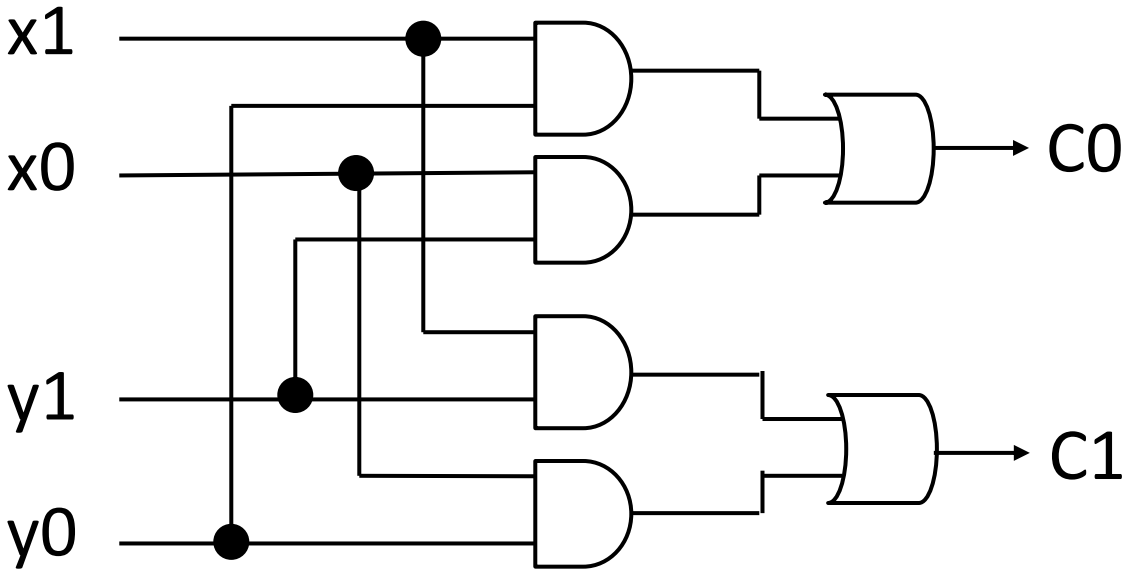
- Duplication/ Comparison Check



Example of Two Rail Code Checker

- 2 output lines
- (C1,C0) = {(0,1),(1,0)} = codeword
- (C1,C0) = {(0,0),(1,1)} = non-codeword

Ex. *2-Pair Two Rail Code Checker:* Input = {x1x0, y1y0}



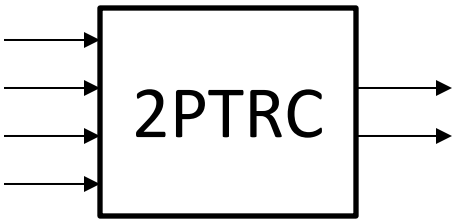
Self Testing Checker

- 2 output lines
- Code-Disjoint
- Self Testing

{0101,0110,1001,1010}

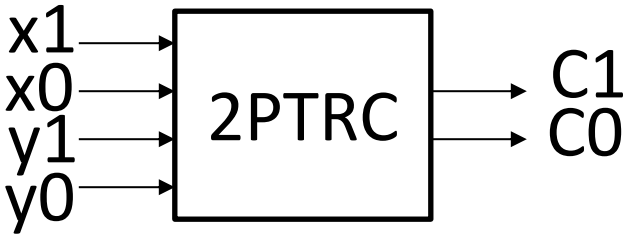
n-Pair Two Rail Code Checker

- n=6 using Tree-Structure *2-Pair Two Rail Code Checker*



Two Rail Code Sequence

Timing/ Sequence	1	2	3	4
R	01	01	10	10
S	01	10	01	10
T	01	10	10	01

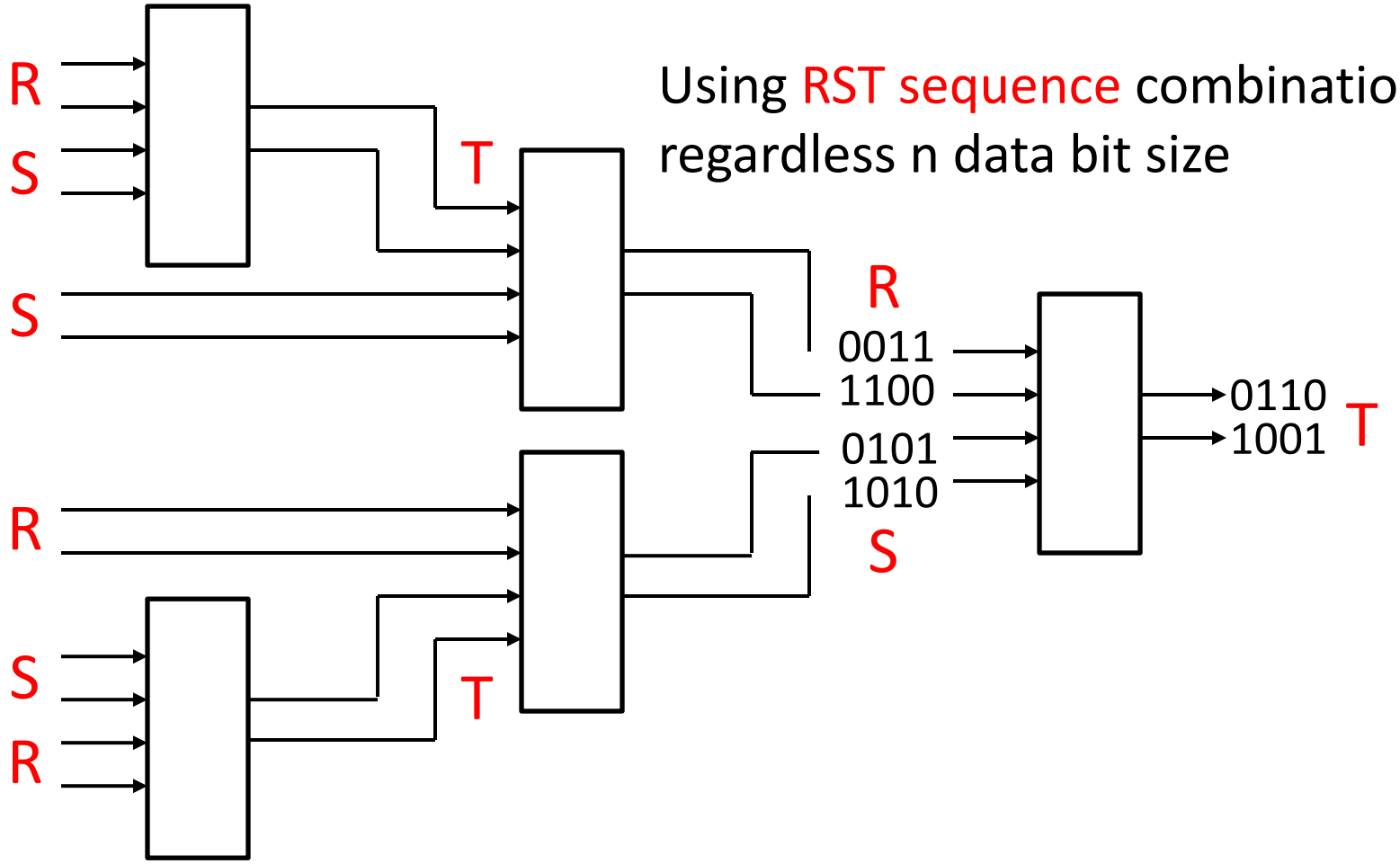


Self Testing for all Stuck-at Faults

IO/ Timing	x1	x0	y1	y0	C1	C0
1	0	1	0	1	0	1
2	0	1	1	0	1	0
3	1	0	0	1	1	0
4	1	0	1	0	0	1

R **S** **T**

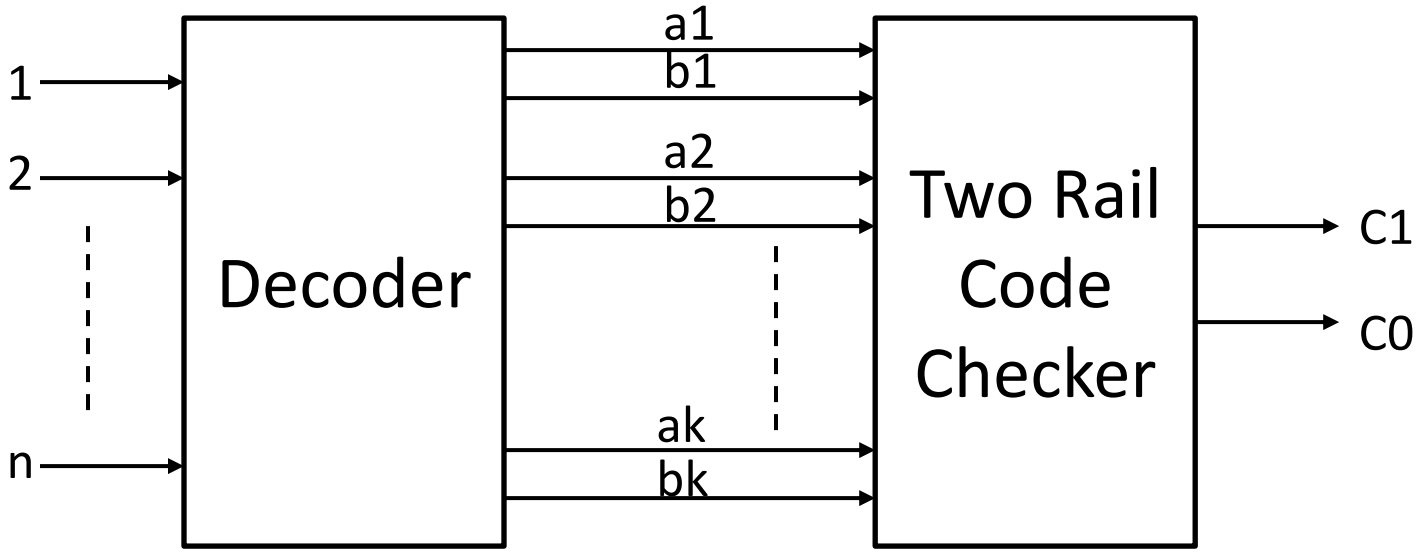
Self Testing for 6-Pair Two Rail Code Checker



Using RST sequence combination regardless n data bit size

1-of-n Code Checker

- Decode n to k $(k = [\log_2 n])$ and connect to Tree-Structure k-Pair Two Rail Code Checkers



Ex. n=8, k=3; Decoder function:

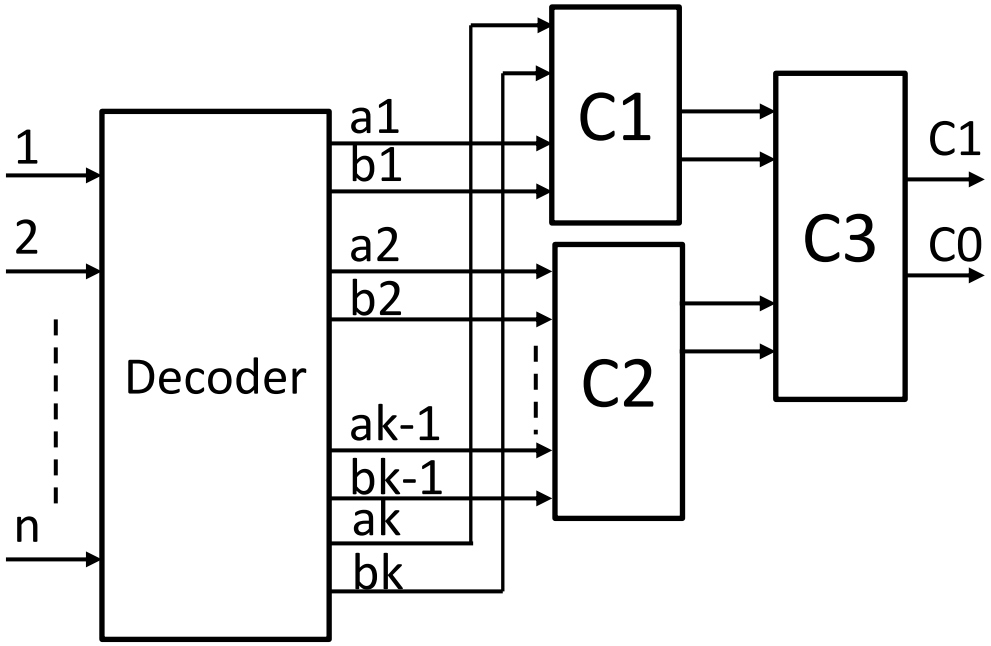
a1 = (1 v 2 v 3 v 4), b1=(5 v 6 v 7 v 8)

a2 = (1 v 2 v 5 v 6), b2=(3 v 4 v 7 v 8)

a3 = (1 v 3 v 5 v 7), b3=(2 v 4 v 6 v 8)

Self Testing 1-of-n Code Checker ($n \neq 2^k$)

- Connect pair (*1,k*) to Two Rail Code Checker (C1) and other pairs to *k-2* Pair Two Rail Code Checker (C2)
- All Checkers have {00,01,10,11} inputs ⇒ Self Testing



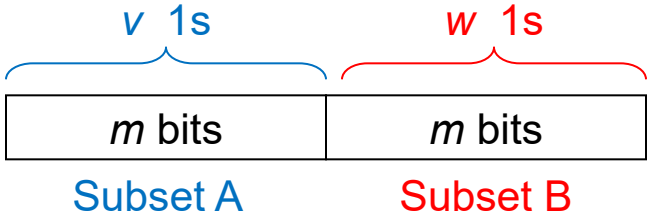
a_i/L_{ip}	a1	a2	…	a_{k-1}	ak
1	0	0	…	0	1
2	0	0	…	1	0
3	0	0	…	1	1
2^{k-1}	1	0	…	0	0
$2^{k-1}+1$	1	0	…	0	1
…	…	…	…	…	…

m-of-2m Code Checker

Divide the $2m$ bits into two disjoint subsets A and B of m bits each
Let v and w be the weight of (number of 1s in) A and B, respectively
Implement the two code checker outputs e_0 and e_1 as follows:

$$C_0 = \bigvee_{\substack{i=0 \\ (i \text{ even})}}^{m} (v \geq i)(w \geq m - i)$$

$$C_1 = \bigvee_{\substack{j=1 \\ (j \text{ odd})}}^{m} (v \geq j)(w \geq m - j)$$



Ex. 3-out-of-6 code checker, $m = 3$, $A = \{a, b, c\}$, $B = \{f, g, h\}$

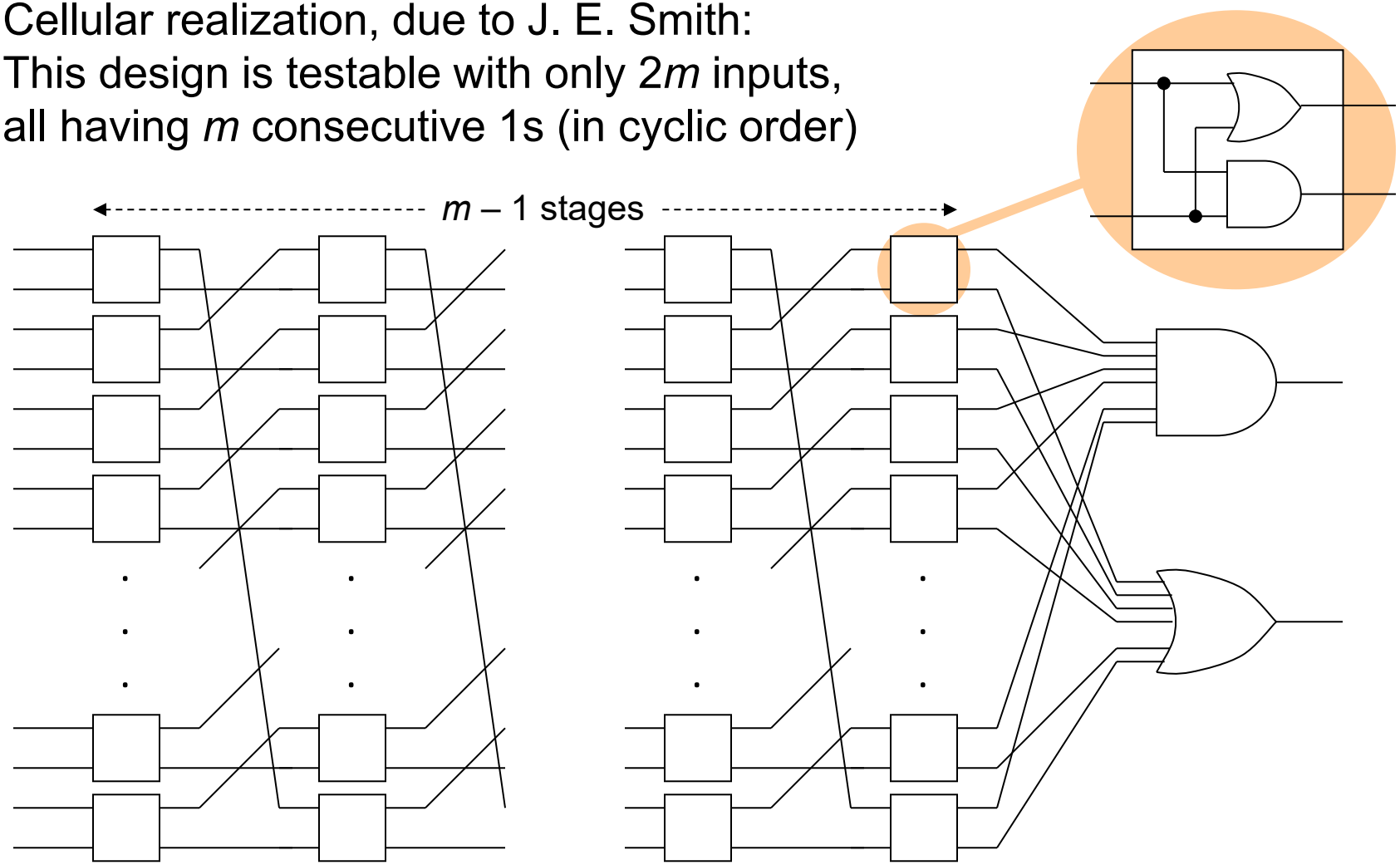
$C_0 = (v \geq 0)(w \geq 3) \vee (v \geq 2)(w \geq 1) = fgh \vee (ab \vee bc \vee ca)(f \vee g \vee h)$

$C_1 = (v \geq 1)(w \geq 2) \vee (v \geq 3)(w \geq 0) = (a \vee b \vee c)(fg \vee gh \vee hf) \vee abc$

Always satisfied

Another TSC *m*-out-of-2*m* Code Checker

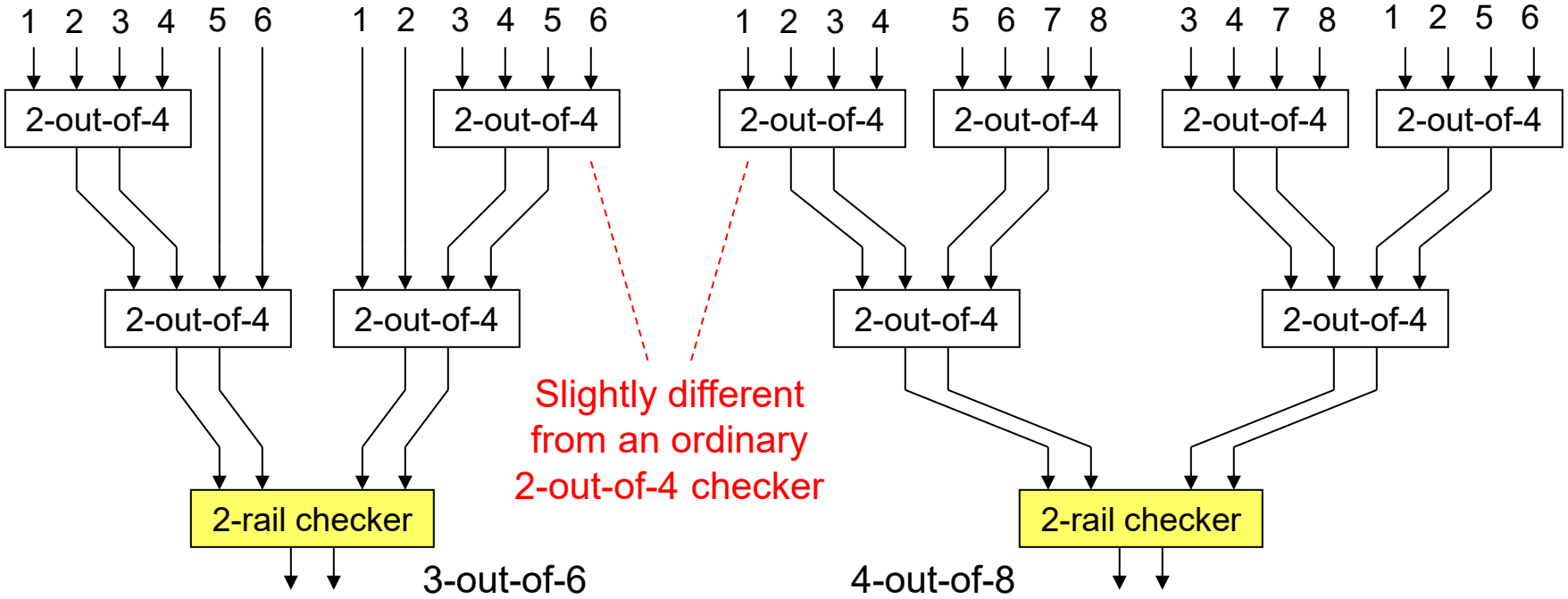
Cellular realization, due to J. E. Smith:
This design is testable with only 2*m* inputs,
all having *m* consecutive 1s (in cyclic order)



Using 2-out-of-4 Checkers as Building Blocks

Building *m*-out-of-2*m* TSC checkers, $3 \leq m \leq 6$, from 2-out-of-4 checkers (construction due to Lala, Busaba, and Zhao):

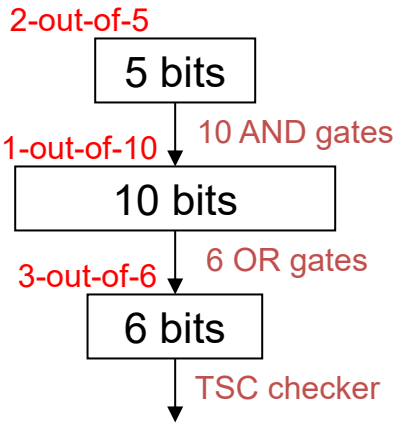
Ex. 3-out-of-6 and 4-out-of-8 TSC checkers are depicted below (only the structure is shown; some design details are missing)



TSC Checker for *k*-out-of-*n* Code

One design strategy is to proceed in 3 stages:
Convert the *k*-out-of-*n* code to a 1-out-of-$\binom{n}{k}$ code
Convert the latter code to an *m*-out-of-2*m* code
Check the *m*-out-of-2*m* code using a TSC checker

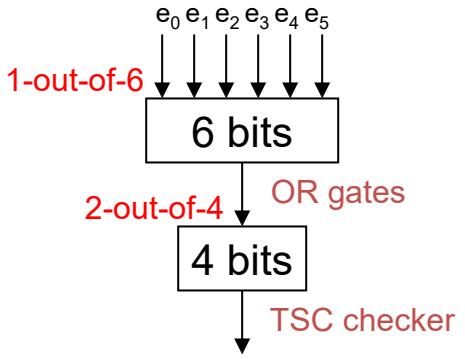
This approach is impractical for many codes



A procedure due to Marouf and Friedman:
Implement 6 functions of the general form →
(these have different subsets of bits as inputs and constitute a 1-out-of-6 code)

$$e_0 = \bigvee_{\substack{j=1 \\ (j \text{ even})}}^{m} (v \geq j)(w \geq m - j)$$

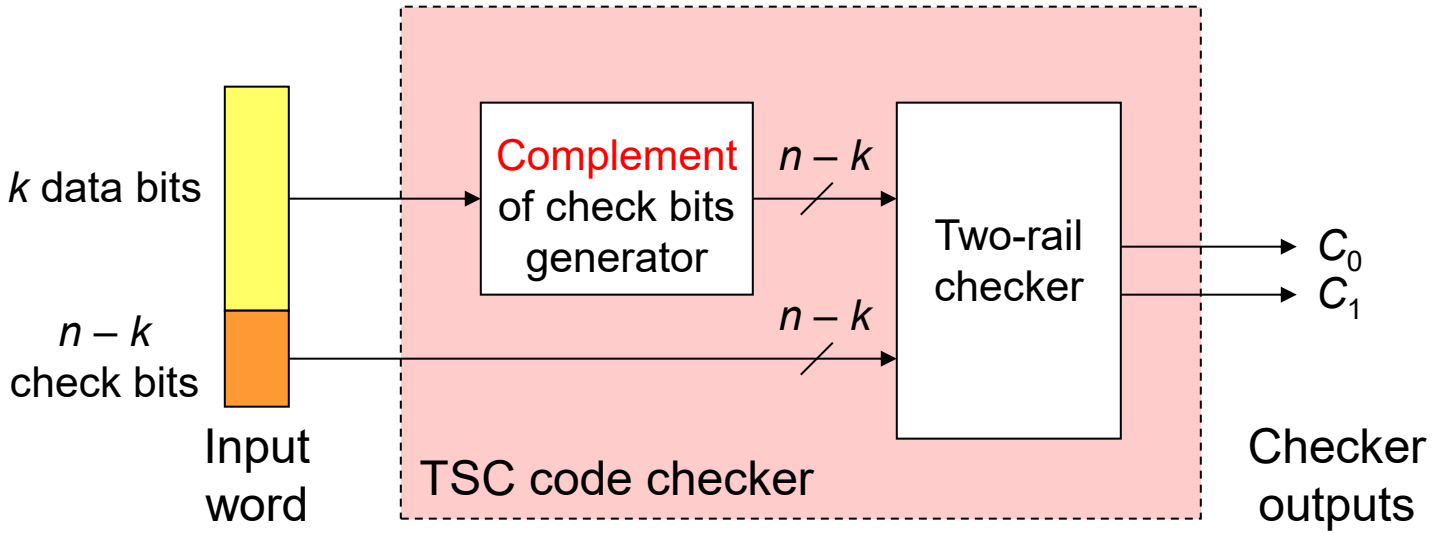
Use a TSC 1-out-of-6 to 2-out-of-4 converter
Use a TSC 2-out-of-4 code checker



The process above works for $2k + 2 \leq n \leq 4k$
It can be somewhat simplified for $n = 2k + 1$

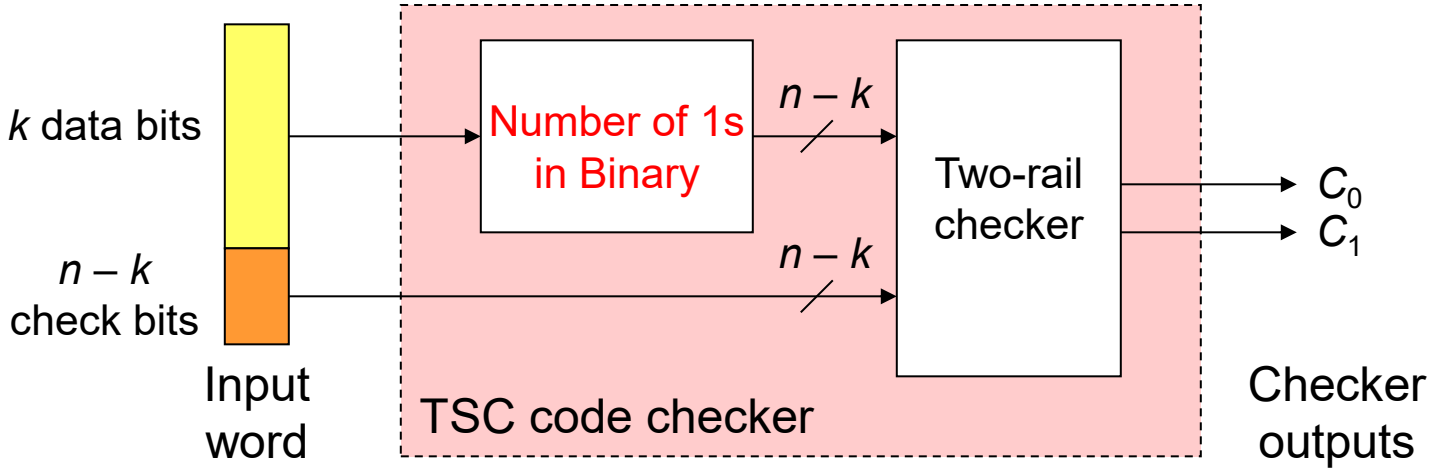
TSC Checker for Separable Codes

Here is a general strategy for designing totally-self-checking checkers for separable codes

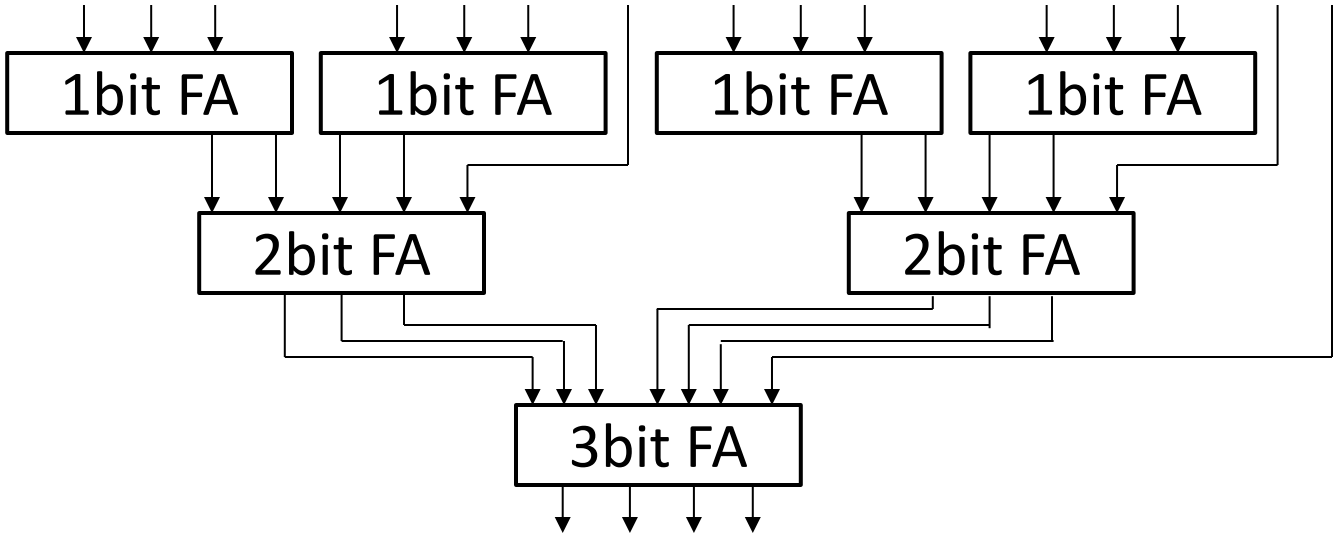


- For many codes, direct synthesis will produce a faster and/or more compact totally-self-checking checker
- Require full combination of Two-rail Checker Inputs ⇒ ST
- No need to consider about ST in Complement of Check Bits Generator

Berger Code Checker

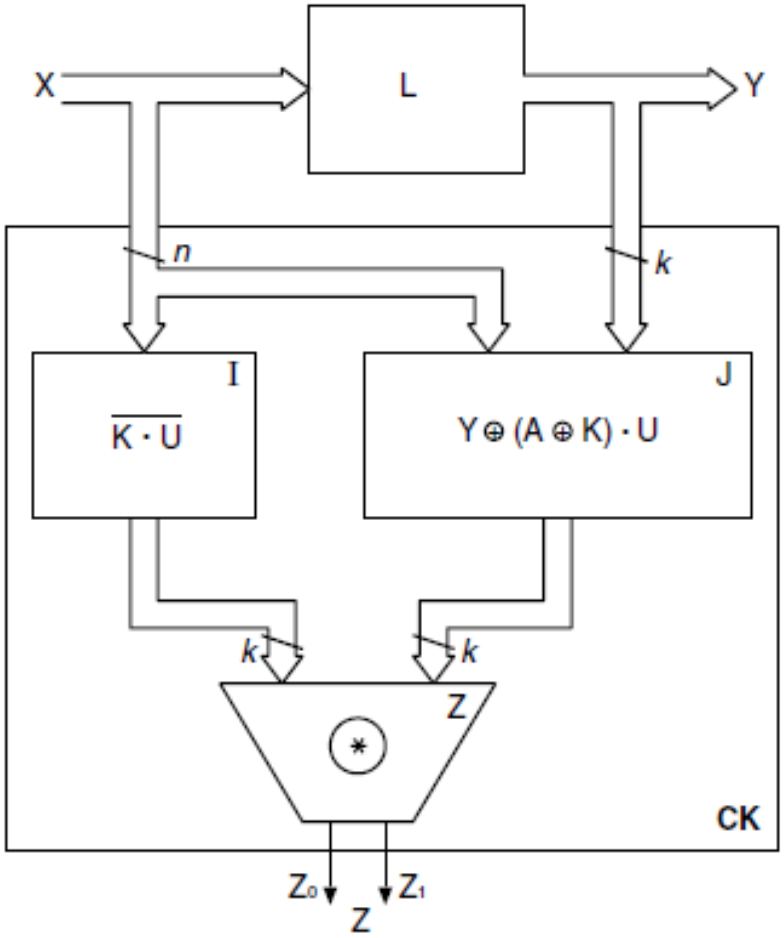


Ex. Complement of Check bit Generator of (15,4) Berger Code



Generalized Prediction Checker (GPC)

- Self-testing checker of the combinational circuits having non-encoded inputs and outputs



TSC Functional Unit

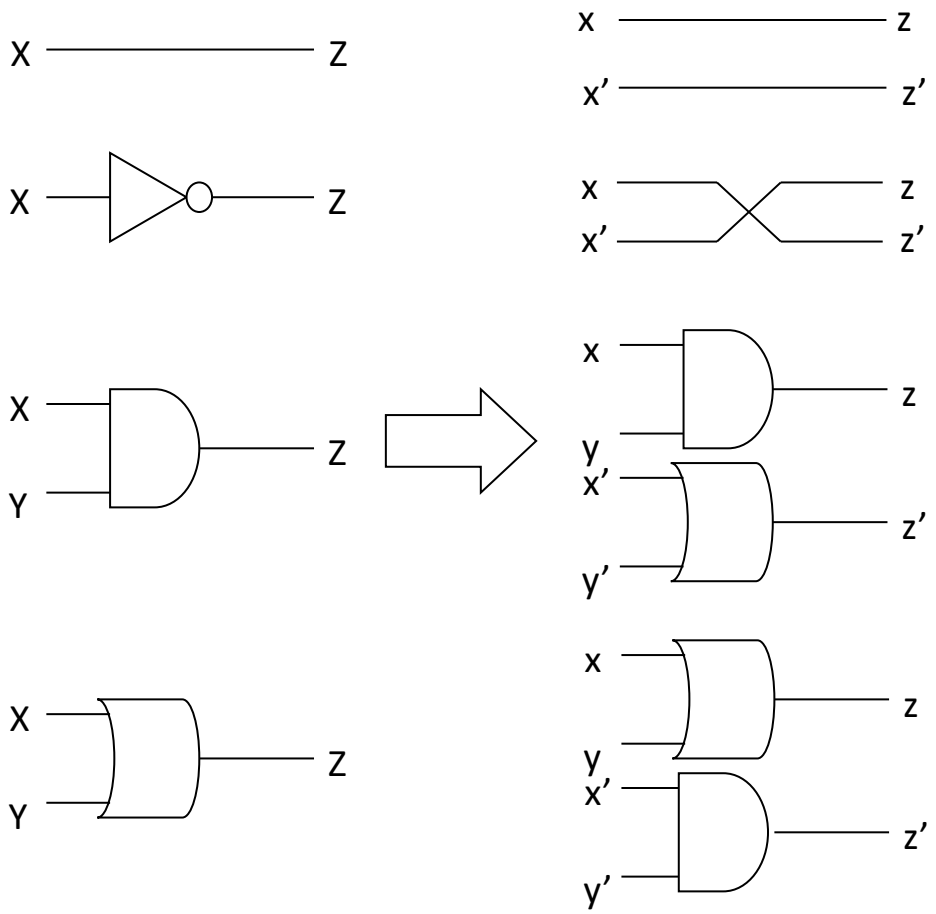
- TSC MUX
  - *Non-inverter parity code data MUX is TSC for unidirectional faults*
- Dual-Rail Logic
  - *Dual-Rail Logic converted from Non-Redundant Single Rail Logic is TSC for unidirectional faults*

TSC Function Unit is Costly!!

Dual Rail Logic

- 1-of-2 Code

  $\{0,1\} \Rightarrow \{01,10\}$
- Easy Implementation (Encoder/Decoder)
- Preserve code after operation
- Non-Inverter Circuit

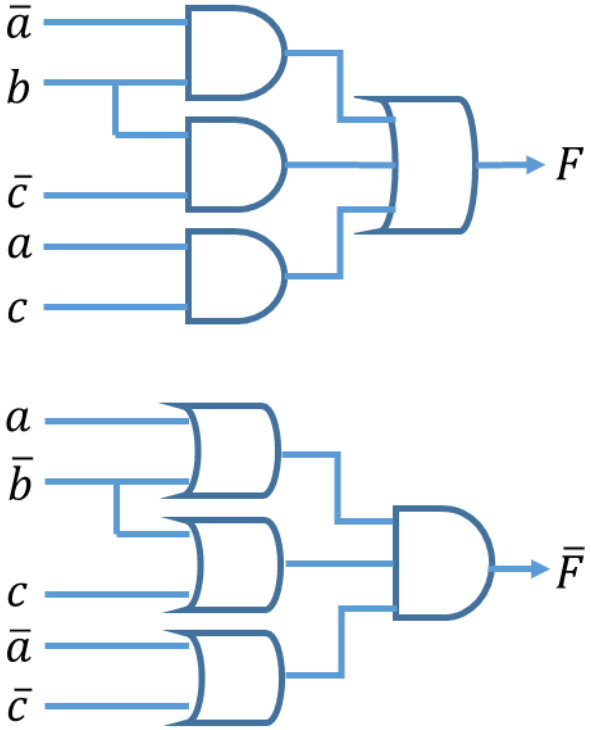


Check Yourself: TSC Functional Unit

- Dual-Logic implementation for

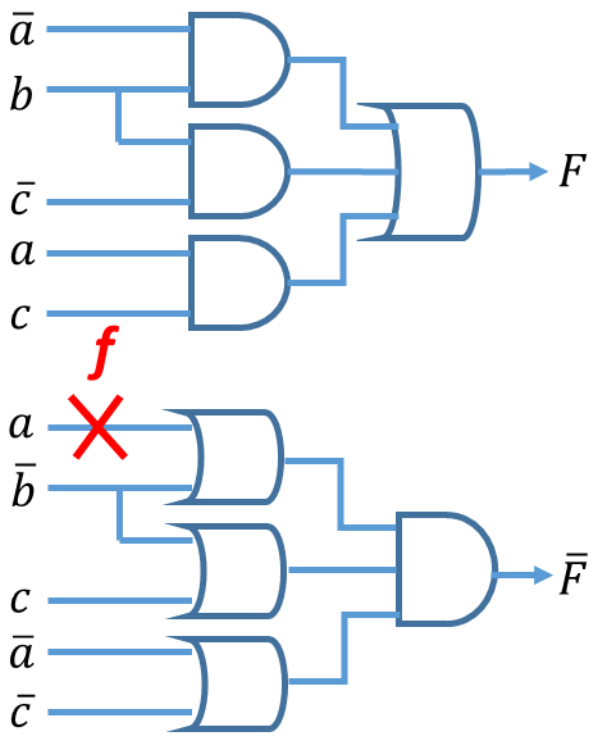
$$F = \bar{a}b + b\bar{c} + ac$$

- Assume one stuck-at-fault,
  - Find the property of each input
  - Conclude the property of circuit



Check Yourself: TSC Functional Unit

- $F = \bar{a}b + b\bar{c} + ac; \bar{F} = (a + \bar{b})(\bar{b} + c)(\bar{a} + \bar{c})$



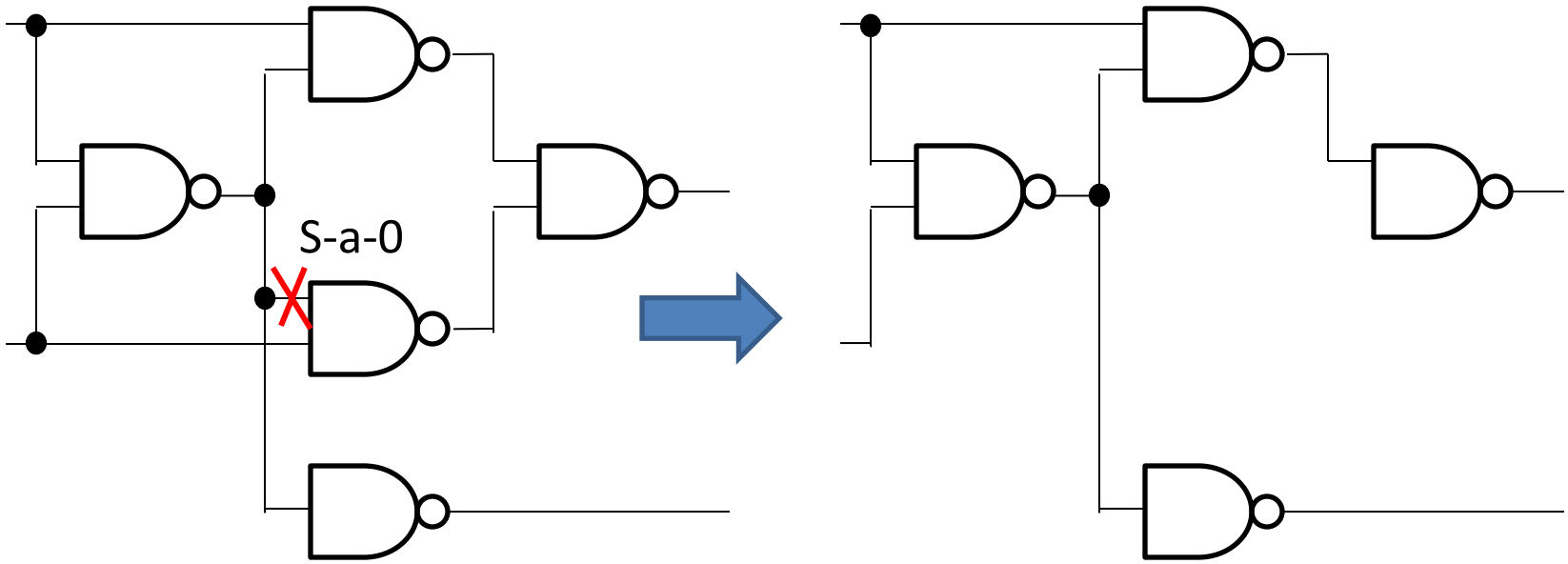
a	$\bar{a}$	b	$\bar{b}$	c	$\bar{c}$	F	$\bar{F}$	f_{s-a-0}
0	1	0	1	0	1	0	1	*same OP*
0	1	0	1	1	0	0	1	*same OP*
0	1	1	0	0	1	1	0	*same OP*
0	1	1	0	1	0	1	0	*same OP*
1	0	0	1	0	1	0	1	*same OP*
1	0	0	1	1	0	0	1	*same OP*
1	0	1	0	0	1	1	0	*non-codeword*
1	0	1	0	1	0	1	0	*non-codeword*

The property of circuit = FS + ST = TSC

Contraction Circuits

Definition: Contraction Circuit G’

- Circuit G’ that remove signal lines and gate outputs which are fixed by a given fault f in the original circuit G



SFS to TSC

Theorem

- Circuit G is SFS for faults F. If no any codeword can detect the fault f, and G' is a contraction circuit of G for the fault f, and F' is an f contraction set of F, then;
  - G' is SFS for F', and
  - When no fault, Both G and G' generate the same outputs against all codeword inputs

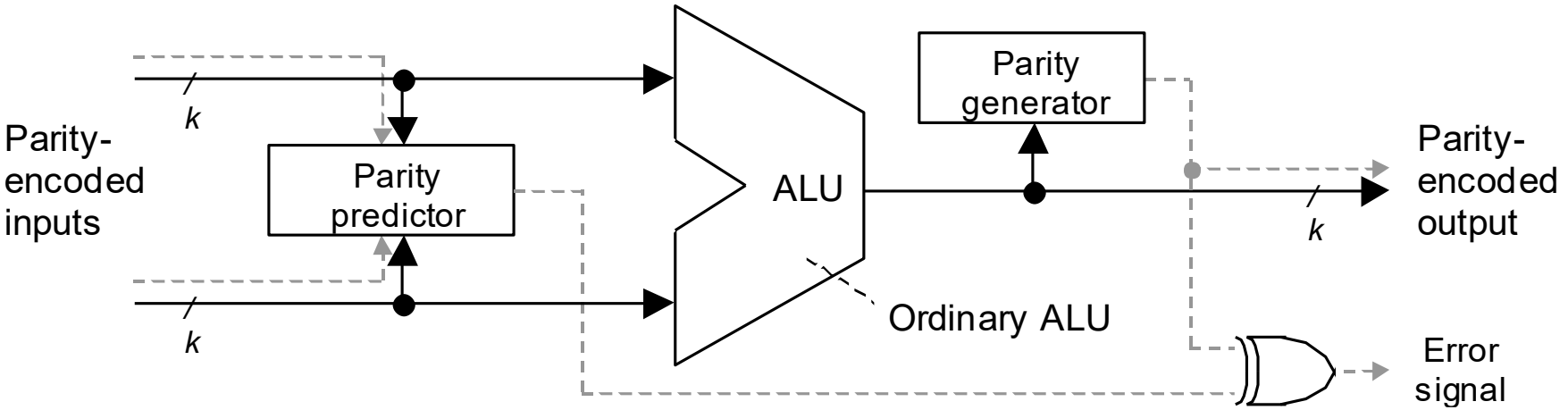
Consider Function Unit as SFS!!

SFS Functional Unit

- Function unit is SFS when;
  - Non-redundant and
  - Non-inverter circuit and
  - Output Coding wt Unordered Codes

TSC Design with Parity Prediction

Parity prediction as an alternative to duplication

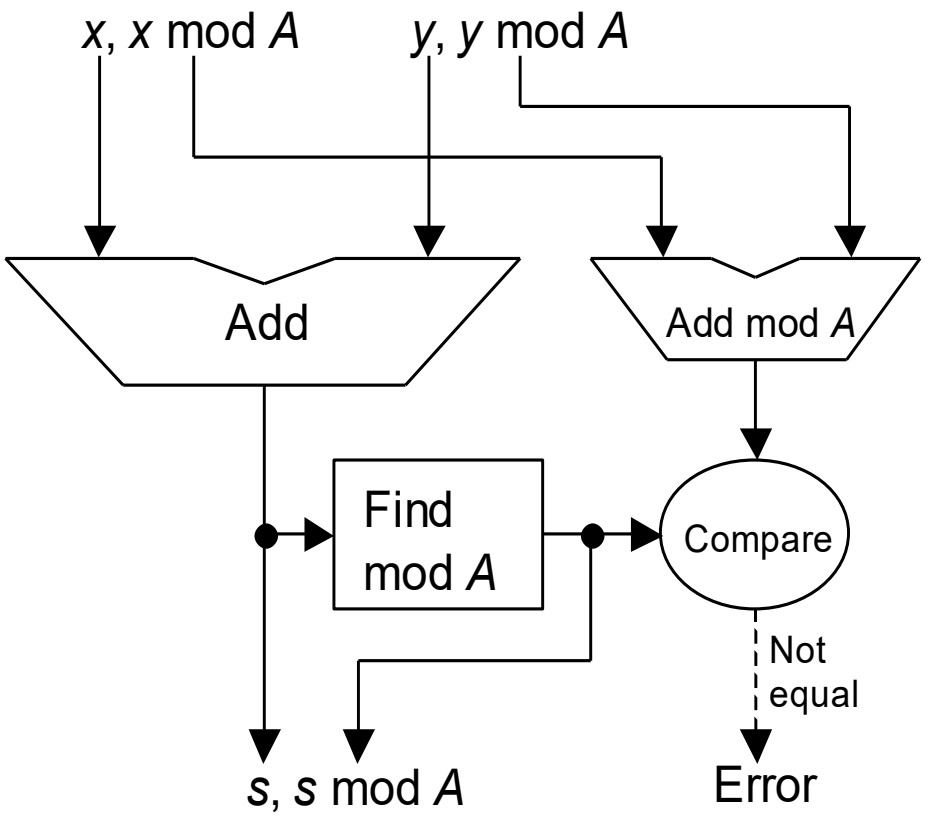


If the parity predictor produces the complement of the output parity, and the XOR gate is removed, we have a self-checking design

To ensure the TSC property, we must also verify that the parity predictor is testable only with input codewords

TSC Design with Residue Encoding

Residue checking is applicable directly to addition, subtraction, and multiplication, and with some extra effort to other arithmetic operations



To make this scheme TSC:

Modify the "Find mod *A*" box to produce the complement of the residue

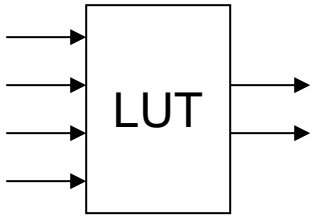
Use two-rail checker instead of comparator

Verify the self-testing property if the residue channel is not completely independent of the main computation (not needed for add/subtract and multiply)

Self-Checking Design with FPGAs

LUT-based FPGAs can suffer from the following fault types:

- Single s-a faults in RAM cells
- Single s-a faults on signal lines
- Functional faults in a multiplexer within a single CLB
- Functional faults in a D flip-flop within a single CLB
- Single s-a faults in pass transistors connecting CLBs



Synthesis algorithm:

(1) Use scripts in the Berkeley synthesis tool SIS to decompose an SOP expression into an optimal collection of parts with 4 or fewer variables

(2) Assign each part to a functional cell that produces a 2-rail output

(3) Connect the outputs of a pair of intermediate functional cells to the inputs of a checker cell and find the output equations for that cell

(4) Cascade the checker cells to form a checker tree

Ref.: P.K. Lala, and A.L. Burress, *IEEE Trans. Instrumentation and Measurement*, Vol. 52, No. 5, pp. 1391-1398, October 2003

Synthesis of TSC Systems from TSC Modules

System consists of a set of modules, with interconnections modeled by a directed graph

Theorem 1: A sufficient condition for a system to be TSC with respect to all single-module failures is to add checkers to the system such that if a path leads from a module M_i to itself (a loop), then it encounters at least one checker

Theorem 2: A sufficient condition for a system to be TSC with respect to all multiple module failures in the module set $A = \{M_i\}$ is to have no loop containing two modules in A in its path and at least one checker in any path leading from one module in A to any other module in A

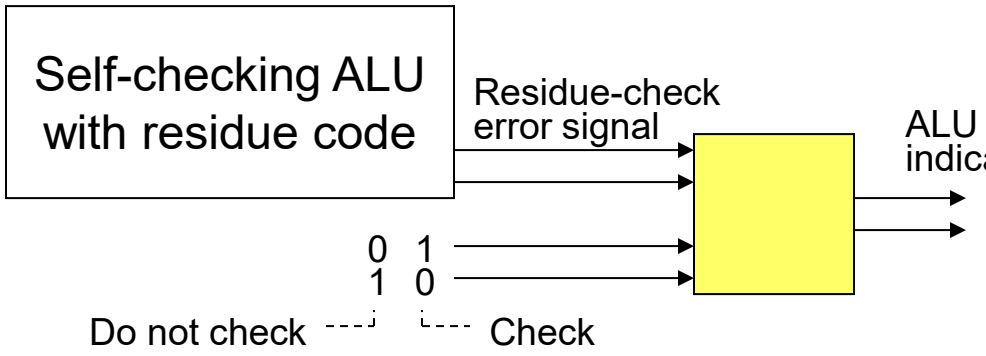
Optimal placement of checkers to satisfy these condition

Easily solved, when checker cost is the same at every interface

Partially Self-Checking Units

Some ALU functions, such as logical operations, cannot be checked using low-redundancy codes

Such an ALU can be made partially self-checking by circumventing the error-checking process in cases where codes are not applicable



01, 10 = G (top)
01 = D, 10 = C (bottom)
00, 11 = B

	In		Out	
	X	B	B	
	B	C	B	
	B	D	G	
Normal operation	G	C	G	01
Normal operation	G	D	G	10

The check/do-not-check indicator is produced by the control unit

Self-Checking State Machines

Design method for Moore-type machines, due to Diaz and Azema:

Inputs and outputs are encoded using two-rail code
States are encoded as *n*/2-out-of-*n* codewords

Fact: If the states are encoded using a *k*-out-of-*n* code, one can express the next-state functions (one for each bit of the next state) via monotonic expressions; i.e., without complemented variables

Monotonic functions can be realized with only AND and OR gates, hence the unidirectional error detection capability

State	Input $x = 0$	Input $x = 1$	Output z
A	*C*	*A*	1
B	*D*	*C*	1
C	*B*	*D*	0
D	*C*	*A*	0

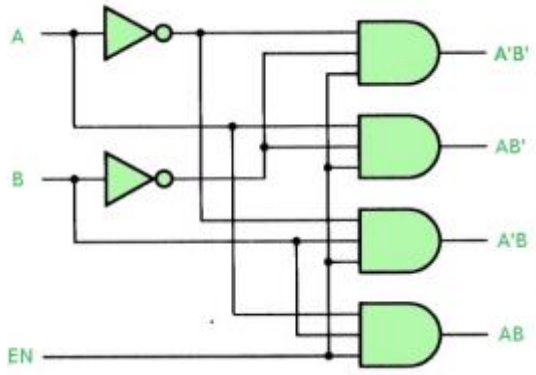
State	Input $x = 01$	Input $x = 10$	Output z
0011	1010	0011	10
0101	1001	1010	10
1010	0101	1001	01
1001	1010	0011	01

Further Studies

- Self-Checking Processor
- Fault Masking Scheme
- System Reconfiguration/Recovery
- Fault Tolerance in Distributed System
- Software Fault Tolerance
- Testing Digital Circuits

HW06: Self Checking Circuits

- *Design a Self-Checking 2:4 Demultiplexer with Enable*



E	A	B	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1